

Paralelismo em nível de instruções e processadores superescalares

14.1 Introdução

- Superescalar *versus* superpipeline
- Limitações

14.2 Questões de projeto

- Paralelismo em nível de instruções e paralelismo de máquina
- Política sobre emissão de instruções
- Renomeação de registradores
- Paralelismo de máquina
- Previsão de desvio
- Execução superescalar
- Implementação superescalar

14.3 Pentium 4

- *Front end*
- Lógica de execução fora de ordem (*out-of-order execution*)
- Unidades de execução de inteiros e de pontos flutuantes

14.4 ARM cortex-A8

- Unidade de busca de instruções
- Unidade de decodificação de instruções
- Unidade de execução de inteiros
- Pipeline SIMD e de ponto flutuante

14.5 Leitura recomendada

PRINCIPAIS PONTOS

- Um **processador superescalar** é aquele em que múltiplos e independentes pipelines de instruções são usados. Cada pipeline consiste de múltiplos estágios, de tal forma que cada pipeline possa lidar com múltiplas instruções ao mesmo tempo. Os pipelines múltiplos introduzem um nível de paralelismo, possibilitando que múltiplos fluxos de instruções sejam processados ao mesmo tempo. Um processador superescalar explora o que é conhecido como **paralelismo em nível de instruções**, o que se refere ao grau em que as instruções de um programa podem ser executadas em paralelo.
- Um processador superescalar normalmente busca múltiplas instruções ao mesmo tempo e depois tenta localizar instruções próximas que sejam independentes umas das outras e, portanto, podem ser executadas em paralelo. Se a entrada de uma instrução depende da saída da instrução anterior, então a instrução posterior não pode completar a execução ao mesmo tempo ou antes da instrução anterior. Uma vez identificadas tais dependências, o processador pode executar e completar instruções em uma ordem diferente do código de máquina original.
- O processador pode eliminar algumas dependências desnecessárias com o uso de registradores adicionais e renomeando referências de registradores no código original.
- Considerando que os processadores RISC puros frequentemente empregam desvios atrasados para maximizar a utilização do pipeline de instruções, este método é menos apropriado para uma máquina superescalar. Em vez disso, a maioria de máquinas superescalares usa a previsão de desvio tradicional para melhorar a eficiência.

Uma implementação superescalar de uma arquitetura de processador é aquela onde as instruções comuns — aritméticas de inteiros e de pontos flutuantes, leituras, escritas e desvios condicionais — podem ser iniciadas simultaneamente e executadas independentemente. Tais implementações levantam uma série de complexas questões de projeto relacionadas ao pipeline de instruções.

O projeto superescalar chegou à cena pouco depois da arquitetura RISC. Embora a arquitetura de conjunto de instruções simplificada de uma máquina RISC leve, por si só, a técnicas superescalares, a abordagem superescalares pode ser usada tanto nas arquiteturas RISC como nas CISC.

Considerando que o período de espera para a chegada de máquinas RISC comerciais desde o início da verdadeira pesquisa RISC com o IBM 801 e o RISC I de Berkeley foi de sete ou oito anos, as primeiras máquinas superescalares se tornaram disponíveis comercialmente dentro de apenas um ou dois anos depois de inventado o termo *superescalar*. A abordagem superescalar se tornou agora o método padrão para implementação de microprocessadores de alto desempenho.

Neste capítulo, começamos com uma introdução à abordagem superescalar, confrontando-a com superpipeline. A seguir, apresentamos as principais questões de projeto associadas com a implementação superescalar. Depois analisamos vários exemplos importantes da arquitetura superescalar.



14.1 Introdução

O termo *superescalar*, criado em 1987 (AGERWALA e COCKE, 1987^a), refere-se a uma máquina que é projetada para melhorar o desempenho da execução de instruções escalares. Na maioria das aplicações, a maior parte das operações é de grandezas escalares. Consequentemente, a abordagem superescalar representa o próximo passo na evolução de processadores de uso geral e de alto desempenho.

A essência da abordagem superescalar é a habilidade de executar instruções independentemente e concorrentemente em pipelines diferentes. O conceito pode ser ainda mais explorado permitindo que as instruções sejam executadas em uma ordem diferente da ordem do programa. A Figura 14.1 mostra, em termos gerais, a abordagem superescalar. Existem muitas unidades funcionais onde cada uma é implementada com um pipeline, o que suporta execução paralela de várias instruções. Neste exemplo, duas instruções de inteiros, duas introduções de pontos flutuantes e uma de memória (leitura ou escrita) podem ser executadas ao mesmo tempo.

Muitos pesquisadores investigaram processadores superescalar e suas pesquisas indicam que algum grau de melhoria de desempenho é possível. A Tabela 14.1 apresenta as vantagens de desempenho reportadas. As diferenças em resultados surgem das diferenças tanto no hardware da máquina simulada quanto das aplicações sendo simuladas.



Superescalar versus superpipeline

Uma abordagem alternativa para alcançar melhor desempenho é conhecida como superpipeline, termo criado em 1988 (JOUPII, 1988^b). O superpipeline explora o fato de que muitos estágios de pipeline executam tarefas que requerem menos do que metade de um ciclo de clock. Assim, a velocidade interna de clock dobrada possibilita o desempenho de duas tarefas em um ciclo de clock externo. Vimos um exemplo desta abordagem com o MIPS R4000.

A Figura 14.2 compara as duas abordagens. A parte superior do diagrama ilustra um pipeline comum, usado como base para comparação. O pipeline base consegue iniciar uma instrução por ciclo de clock e pode executar um estágio de pipeline por ciclo de clock.

O pipeline tem quatro estágios: busca instrução, decodificação da operação, executar operação e atualização do resultado. O estágio de execução é preenchido com linhas cruzadas para facilidade de compreensão. Observe

Figura 14.1 Organização superescalar geral

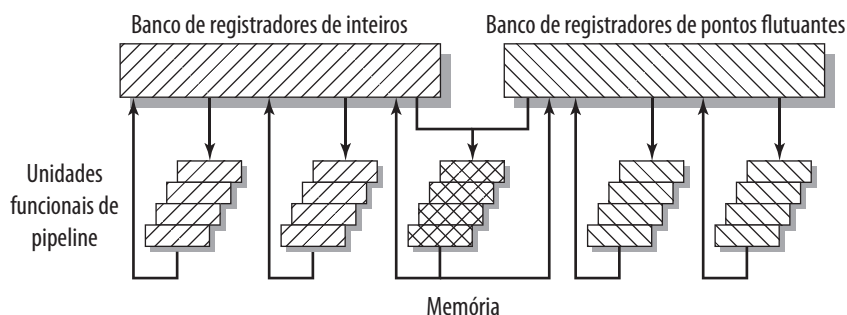
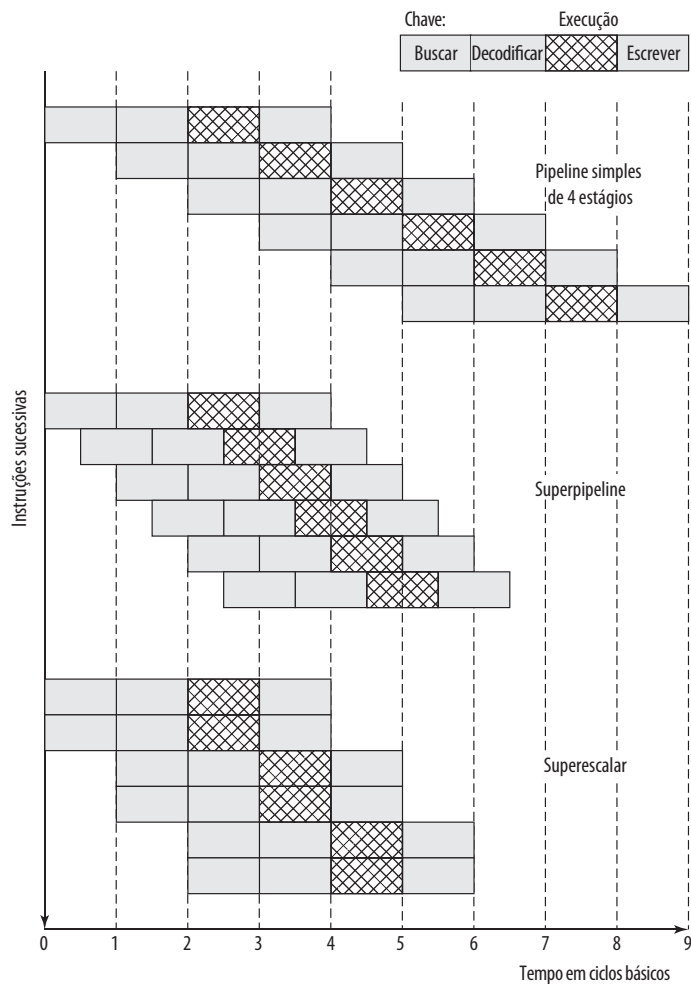


Tabela 14.1 Acelerações reportadas das máquinas do tipo superescalar

Referência	Aumento de velocidade (<i>speed up</i>)
Tjaden e Flynn, 1970 ^c	1,8
Kuck, Parker e Sameh, 1977 ^d	8
Weiss e Smith, 1984 ^e	1,58
Acosta, Kjelstrup e Torng, 1986 ^f	2,7
Sohi, 1990 ^g	1,8
Smith, Johnson e Horowitz, 1989 ^h	2,3
Jouppi, 1989 ⁱ	2,2
Lee, Kwok e Briggs, 1991 ^j	7

Figura 14.2 Comparação da abordagem superescalar e superpipeline



que, embora várias instruções estejam sendo executadas concorrentemente, apenas uma está no seu estágio de execução a qualquer tempo.

A próxima parte do diagrama mostra uma implementação de superpipeline capaz de executar dois estágios de pipeline por ciclo de clock. Uma forma alternativa de analisar isso é que as funções executadas em cada estágio podem ser divididas em duas partes que não se sobrepõem e cada uma pode ser executada na metade de ciclo de clock. Uma implementação de superpipeline que se comporta desta forma é considerada como de nível 2. Finalmente, a parte mais baixa do diagrama mostra uma implementação superescalares capaz de executar duas instâncias de cada estágio em paralelo. Implementações de superpipeline e as superescalares de níveis mais altos também são possíveis.

Tanto a implementação de superpipeline como a superescalar ilustradas na Figura 14.2 possuem o mesmo número de instruções executando ao mesmo tempo em determinado estado. O processador de superpipeline fica para trás do processador superescalar no início do programa e a cada alvo de desvio.



Limitações

A abordagem superescalar depende da habilidade de executar múltiplas instruções em paralelo. O termo **paralelismo em nível de instruções** refere-se ao grau em que, em média, as instruções de um programa podem ser executadas em paralelo. Uma combinação de otimização baseada em compilador e técnicas de hardware pode ser usada para maximizar o paralelismo em nível de instruções. Antes de analisar as técnicas de design usadas em máquinas superescalares para aumentar o paralelismo em nível de instruções, precisamos olhar as limitações fundamentais do paralelismo com as quais o sistema deve lidar. Johnson (1991^k) apresenta cinco limitações:

- Dependência de dados verdadeira.
- Dependência procedural.
- Conflitos de recursos.
- Dependência de saída.
- Antidependência.

Analisamos as primeiras três destas limitações no restante desta seção. Uma discussão sobre as duas últimas precisa aguardar o desenrolar da próxima seção.

DEPENDÊNCIA DE DADOS VERDADEIRA Considere a seguinte sequência:¹

```
ADD    EAX,    ECX    ;carrega o registrador EAX com o conteúdo
                        de ECX mais o conteúdo de EAX
MOV    EBX,    EAX    ;carrega EBX com o conteúdo de EAX
```

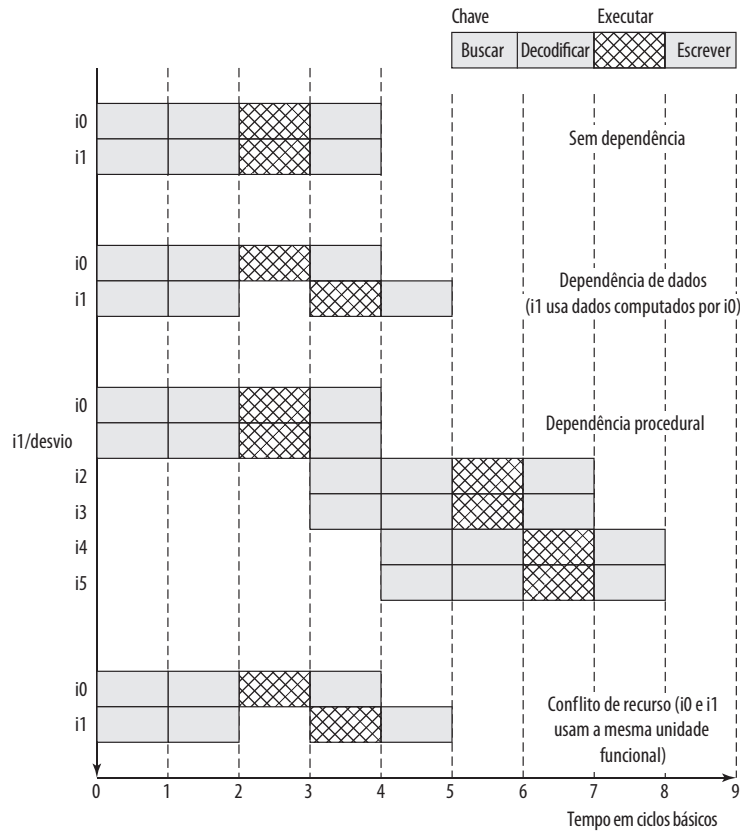
A segunda instrução pode ser obtida e decodificada, mas não pode ser executada até que a primeira execute. O motivo é que a segunda instrução precisa de dados produzidos pela primeira. Esta situação é conhecida como **dependência de dados verdadeira** (também chamada de **dependência de fluxo** ou dependência de **escrita após leitura [WAR, do inglês write after read]**).

A Figura 14.3 do inglês esse dependência em uma máquina superescalar de nível 2. Sem dependências, duas instruções podem ser obtidas e executadas em paralelo. Se houver uma dependência de dados entre a primeira e a segunda instrução, então a segunda é atrasada por tantos ciclos de clock quantos forem necessários para remover a dependência. Em geral, qualquer instrução tem que ser atrasada até que todos os seus valores de entrada tenham sido produzidos.

Em um pipeline simples, como o mostrado na parte superior da Figura 14.2, a sequência de instruções acima mencionada não causaria nenhum atraso. No entanto, considere o seguinte, onde uma das leituras ocorre na memória em vez de um registrador:

```
MOV    EAX,    eff    ;carregar registrador EAX com conteúdo
                        do endereço de memória efetivo eff
MOV    EBX,    EAX    ;carregar EBX com conteúdo de EAX
```

¹ Para linguagem de montagem Intel x86, o símbolo “;” (ponto e vírgula) inicia um campo de comentário.

Figura 14.3 Efeito das dependências

Um processador RISC típico leva dois ou mais ciclos para executar uma leitura de memória quando a leitura é um acesso à cache. Pode levar dezenas ou até centenas de ciclos para uma falha de cache em todos os níveis de cache por causa do atraso de um acesso à memória fora do chip. Uma maneira de compensar este atraso é o compilador reordenar as instruções para que uma ou mais instruções subsequentes que não dependem da leitura de memória possam seguir pelo pipeline. Este esquema é menos eficiente no caso de um pipeline superescalar: as instruções independentes executadas durante a leitura provavelmente serão executadas no primeiro ciclo da leitura, deixando o processador sem fazer nada até que a leitura complete.

DEPENDÊNCIAS PROCEDURAIS Conforme discutimos no Capítulo 12, a presença de desvios em uma sequência de instruções complica a operação do pipeline. As instruções que vêm depois de um desvio (tomado ou não) possuem uma dependência procedural com o desvio e não podem ser executadas até que o desvio seja executado. A Figura 14.3 ilustra o efeito de um desvio em um pipeline superescalar de nível 2.

Como já vimos, este tipo de dependência procedural afeta também um pipeline escalar. A consequência para um pipeline superescalar é mais severa, porque uma oportunidade de magnitude maior é perdida com cada atraso.

Se as instruções de tamanho variável forem usadas, então outro tipo de dependência procedural surge. Como o tamanho de nenhuma instrução particular é conhecido, ela deve ser pelo menos parcialmente decodificada antes que a próxima possa ser obtida. Esta é uma das razões por que as técnicas superescalares são mais prontamente aplicáveis para uma arquitetura RISC, ou uma parecida com RISC, com seu tamanho fixo de instruções.

CONFLITO DE RECURSOS Um conflito de recursos é uma competição de duas ou mais instruções pelo mesmo recurso e ao mesmo tempo. Exemplos de recursos incluem memória, cache, barramentos, entradas para banco de registradores e unidades funcionais (por exemplo, somador da ALU).

Em termos de pipeline, um conflito de recurso mostra o comportamento parecido de uma dependência de dados (Figura 14.3). No entanto, há algumas diferenças. Para começar, os conflitos de recursos podem ser evitados com a duplicação de recursos, considerando que uma dependência de dados verdadeira não pode ser eliminada. Além disso, quando uma operação leva muito tempo para se completar, conflitos de recursos podem ser minimizados pelo pipeline na unidade funcional apropriada.



14.2 Questões de projeto



Paralelismo em nível de instruções e paralelismo de máquina

Jouppi e Wall (1989) fazem uma distinção importante entre dois conceitos relacionados: paralelismo em nível de instruções e paralelismo de máquina. O **paralelismo em nível de instruções** existe quando as instruções de uma sequência são independentes e, assim, podem ser executadas em paralelo por sobreposição.

Como um exemplo do conceito de paralelismo, considere dois pedaços de código a seguir (JOU PPI, 1989):

```
Load R1 ← R2      Add R3 ← R3, "1"
Add R3 ← R3, "1"   Add R4 ← R3, R2
Add R4 ← R4, R2     Store [R4] ← R0
```

As três instruções à esquerda são independentes e, na teoria, todas poderiam ser executadas em paralelo. Ao contrário disso, as três instruções à direita não podem ser executadas em paralelo porque a segunda instrução usa o resultado da primeira e a terceira usa o resultado da segunda.

O grau do paralelismo em nível de instruções é determinado pela frequência da dependência de dados verdadeira e das dependências procedurais no código. Estes fatores, por sua vez, são dependentes da arquitetura do conjunto de instruções e da aplicação. O paralelismo em nível de instruções é também determinado pelo que Jouppi e Wall (1989) denominam como latência da operação: o tempo até que o resultado de uma operação esteja disponível para uso como um operando em uma instrução subsequente. A latência determina quanto atraso uma dependência de dados ou procedural irá causar.

O **paralelismo de máquina** é uma medida da habilidade do processador para obter vantagem do paralelismo em nível de instruções. Ele é determinado pelo número de instruções que podem ser obtidas e executadas ao mesmo tempo (o número de pipelines paralelos) e pela velocidade e sofisticação dos mecanismos que o processador usa para localizar instruções independentes.

O paralelismo em nível de instruções e o de máquina são fatores importantes para melhorar o desempenho. Um programa pode não ter paralelismo em nível de instruções suficiente para obter a vantagem total do paralelismo de máquina. O uso de uma arquitetura com conjunto de instruções de tamanho fixo, como em RISC, melhora o paralelismo em nível de instruções. Por outro lado, o paralelismo de máquina limitado irá limitar o desempenho, não se importando com qual a natureza do programa.



Política de emissão de instruções

Conforme mencionado, o paralelismo de máquina não é simplesmente uma questão de ter múltiplas instâncias de cada estágio do pipeline. O processador também deve ser capaz de identificar o paralelismo em nível de máquina e orquestrar a busca, a decodificação e a execução de instruções em paralelo. Johnson (1991^b) usa o termo **emissão da instrução (instruction issue)** para se referir ao processo de iniciação da execução da instrução em unidades funcionais do processador e o termo **política de emissão de instruções** para se referir ao protocolo usado para a emissão de instruções. Em geral, podemos dizer que a emissão da instrução ocorre quando a instrução é movida do estágio de decodificação para o primeiro estágio de execução do pipeline.

Basicamente, o processador está tentando olhar para frente do ponto atual de execução para localizar instruções que podem ser trazidas no pipeline e executadas. Três tipos de ordenação são importantes nesta consideração:

- A ordem em que as instruções são lidas.
- A ordem em que as instruções são executadas.
- A ordem em que as instruções atualizam o conteúdo dos registradores e as posições de memória.

Quanto mais sofisticado o processador, menos ele é ligado por um relacionamento estrito entre essas ordens. Para otimizar a utilização de vários elementos do pipeline, o processador precisa alterar uma ou mais dessas ordens respeitando a ordem a ser encontrada em uma execução estritamente sequencial. Uma restrição do processador é que o resultado tem que ser correto. Assim, o processador deve acomodar várias dependências e conflitos discutidos anteriormente.

Em termos gerais, podemos agrupar políticas de emissão de instruções superescalares em seguintes categorias:

- Emissão em-ordem com conclusão em-ordem.
- Emissão em-ordem com conclusão fora-de-ordem.
- Emissão fora-de-ordem com conclusão fora-de-ordem.

EMISSÃO EM-ORDEM COM CONCLUSÃO EM-ORDEM A política mais simples de emissão de instruções é a ordem exata que seria alcançada pela execução sequencial (emissão em-ordem) e pela escrita de resultados na mesma ordem (conclusão em-ordem). Nem mesmo pipelines escalares seguem uma política tão simplista. No entanto, é útil considerar esta política como uma base de comparação para abordagens mais sofisticadas.

A Figura 14.4a oferece um exemplo dessa política. Supomos um pipeline superescalar capaz de obter e decodificar duas instruções ao mesmo tempo, com três unidades funcionais (por exemplo, duas aritméticas de inteiros

Figura 14.4 Políticas de emissão e conclusão de instruções superescalares

Decodificar		Executar			Escrever		Ciclo
I1	I2						1
I3	I4	I1	I2				2
I3	I4	I1					3
	I4			I3	I1	I2	4
I5	I6			I4			5
	I6		I5		I3	I4	6
			I6				7
					I5	I6	8

(a) Emissão em-ordem e conclusão em-ordem

Decodificar		Executar			Escrever		Ciclo
I1	I2						1
I3	I4	I1	I2				2
	I4	I1		I3	I2		3
I5	I6			I4	I1	I3	4
	I6		I5		I4		5
			I6		I5		6
					I6		7

(b) Emissão em-ordem e conclusão fora-de-ordem

Decodificar		Janela	Executar			Escrever		Ciclo
I1	I2							1
I3	I4	I1, I2	I1	I2				2
I5	I6	I3, I4	I1		I3	I2		3
		I4, I5, I6		I6	I4	I1	I3	4
		I5		I5		I4	I6	5
						I5		6

(c) Emissão fora-de-ordem e conclusão fora-de-ordem

e uma aritmética de ponto flutuante) e duas instâncias de estágio de escrita no pipeline. O exemplo assume as seguintes restrições em um fragmento de código de seis instruções:

- I1 requer dois ciclos para executar.
- I3 e I4 competem pela mesma unidade funcional.
- I5 depende do valor produzido por I4.
- I5 e I6 competem pela mesma unidade funcional.

Duas instruções são obtidas ao mesmo tempo e passadas para a unidade de decodificação. Como as instruções são obtidas em pares, as duas próximas instruções precisam esperar até que o par de estágios de decodificação do pipeline esteja limpo. Para garantir a conclusão em-ordem, quando ocorre um conflito por uma unidade funcional ou quando uma unidade funcional requer mais do que um ciclo para gerar um resultado, a emissão da instrução para temporariamente.

Neste exemplo, o tempo gasto desde a decodificação da primeira instrução até a escrita dos últimos resultados é de oito ciclos.

EMISSÃO EM ORDEM COM CONCLUSÃO FORA-DE-ORDEM A conclusão fora-de-ordem é usada em processadores RISC escalar para melhorar o desempenho das instruções que requerem múltiplos ciclos. A Figura 14.4b ilustra o seu uso em um processador superescalar. A instrução I2 tem permissão para concluir antes da I1. Isso permite que a I3 seja completada antes, com resultando na economia de um ciclo.

Com conclusão fora-de-ordem, qualquer número de instruções pode estar no estágio de execução em qualquer tempo até o nível máximo do paralelismo da máquina através de todas as unidades funcionais. A emissão de instruções é parada por um conflito de recurso, uma dependência de dados ou uma dependência procedural.

Além das limitações acima mencionadas, uma nova dependência, a qual chamamos anteriormente de **dependência de saída** (também chamada de **dependência de escrita após escrita (WAW, do inglês write after write)**), aparece. O seguinte pedaço de código ilustra esta dependência (*op* representa qualquer operação):

```
I1: R3 ← R3 op R5
I2: R4 ← R3 + 1
I3: R3 ← R5 + 1
I4: R7 ← R3 op R4
```

A instrução I2 não pode executar antes da instrução I1, porque ela precisa do resultado no registrador R3 produzido por I1; este é um exemplo de uma dependência de dados verdadeira, conforme descrito na Seção 14.1. De forma semelhante, I4 precisa esperar pela I3 porque ela usa um resultado produzido por I3. E sobre o relacionamento entre I1 e I3? Aqui não há dependência, conforme definimos. No entanto, se I3 conclui a execução depois de I1, então o valor errado do conteúdo de R3 será obtido para execução de I4. Consequentemente, I3 precisa completar antes de I1 para produzir os valores de saída corretos. Para garantir isso, a emissão da terceira instrução deve ser parada se o seu resultado pode ser sobrescrito posteriormente por uma instrução mais antiga que leva mais tempo para completar.

A conclusão fora-de-ordem requer uma lógica de emissão de instruções mais complexa do que a conclusão em-ordem. Além disso, é mais difícil lidar com interrupções e exceções de instruções. Quando uma interrupção ocorre, a execução da instrução atual é suspensa, para ser continuada depois. O processador deve garantir que a continuação leva em conta que, no momento da interrupção, as instruções à frente da instrução que causou a interrupção já devem ter sido completadas.

EMISSÃO FORA-DE-ORDEM COM CONCLUSÃO FORA-DE-ORDEM Com emissão em-ordem, o processador irá apenas decodificar as instruções até o ponto de uma dependência ou conflito. Nenhuma instrução adicional é decodificada até que o conflito seja resolvido. Como resultado, o processador não pode procurar à frente do ponto de conflito pelas instruções subsequentes que podem ser independentes daquelas que já estão no pipeline e que podem ser útilmente introduzidas no pipeline.

Para permitir emissão fora-de-ordem, é necessário separar os estágios de decodificação e execução do pipeline. Isso é feito com um buffer conhecido como **janela de instruções**. Com esta organização, depois que o processador termina de decodificar uma instrução, ela é colocada na janela de instruções. Enquanto este buffer não estiver cheio, o processador pode continuar a obter e decodificar novas instruções. Quando uma unidade funcional se torna disponível no estágio de execução, uma instrução da janela de instruções pode ser emitida para o estágio de execução. Qualquer instrução pode ser emitida considerando que (1) ela precisa de uma unidade funcional particular que esteja disponível e (2) que nenhum conflito ou dependência bloqueie esta instrução.

O resultado desta organização é que o processador tem a capacidade de olhar para frente, o que permite que ele identifique instruções independentes que podem ser trazidas para o estágio de execução. As instruções são emitidas a partir da janela de instruções sem se preocupar com a ordem do programa original. Assim como antes, a única restrição é que a execução do programa se comporte corretamente.

A Figura 14.4c ilustra essa política. Durante cada um dos três primeiros ciclos, duas instruções são obtidas para o estágio de decodificação. Durante cada ciclo, sujeito a restrição do tamanho do buffer, duas instruções são movidas do estágio de decodificação para a janela de instruções. Neste exemplo é possível emitir a instrução I6 na frente de I5 (lembre que I5 depende de I4, mas I6 não). Assim, um ciclo é economizado no estágio de execução e no de escrita, e a economia total, se comparada à Figura 14.4b, é de um ciclo.

A janela de instruções é apresentada na Figura 14.4c para ilustrar o seu papel. No entanto, essa janela não é um estágio adicional do pipeline. Uma instrução estando na janela simplesmente significa que o processador tem informação suficiente sobre essa instrução para decidir quando ela pode ser emitida.

A política de emissão fora-de-ordem e conclusão fora-de-ordem é sujeita às mesmas restrições descritas anteriormente. Uma instrução não pode ser emitida se violar uma dependência ou um conflito. A diferença é que mais instruções estão disponíveis para emissão, reduzindo a probabilidade que um estágio do pipeline tenha que parar. Além disso, uma nova dependência que anteriormente chamamos de **antidependência** (também chamada de **dependência ler após escrever (RAW, do inglês read after write)**) aparece. O pedaço de código considerado anteriormente ilustra essa dependência:

```
I1: R3 ← R3 op R5
I2: R4 ← R3 + 1
I3: R3 ← R5 + 1
I4: R7 ← R3 op R4
```

A instrução I3 não pode completar a execução antes de instrução I2 começar a executar e obter os seus operandos. Isso é assim porque I3 atualiza o registrador R3, o qual é o operando de origem para I2. O termo *antidependência* é usado porque a restrição é semelhante àquela da dependência de dados verdadeira, só que inversa: em vez de a primeira instrução produzir um valor que é usado pela segunda, a segunda instrução destrói o valor que é usado pela primeira instrução.



Simulador de reordenação de buffer
Simulador de algoritmo de Tomasulo
Simulação alternativa do algoritmo de Tomasulo

Uma técnica comum, usada para suportar conclusão fora-de-ordem, é o buffer de reordenação. Trata-se de um armazenamento temporário para resultados completados fora de ordem que depois são colocados no banco de registradores na ordem do programa. Um conceito relacionado é o algoritmo de Tomasulo. O Apêndice I analisa esses conceitos.



Renomeação de registradores

Quando a emissão de instruções fora-de-ordem e/ou conclusão de instruções fora-de-ordem são permitidas, nós vimos que aumenta a possibilidade de dependências WAW e WAR. Estas dependências diferem das dependências de dados RAW e dos conflitos de recursos que refletem o fluxo de dados através de um programa e sequência da execução. As dependências WAW e WAR, por outro lado, surgem porque os valores nos registradores podem não refletir mais a sequência de valores definida pelo fluxo do programa.

Quando as instruções são emitidas e completadas na sequência, é possível especificar o conteúdo de cada registrador em cada ponto da execução. Quando as técnicas fora-de-ordem são usadas, os valores nos registradores não podem ser totalmente conhecidos em cada momento apenas considerando a sequência de instruções definida pelo programa. Na verdade, os valores estão em conflito pelo uso de registradores e o processador precisa resolver esses conflitos parando ocasionalmente um estágio do pipeline.

As antidependências e dependências de saída são exemplos de conflitos de armazenamento. As instruções múltiplas competem pelo uso de mesmas posições de registradores, gerando restrições de pipeline que retardam o desempenho. O problema se torna mais sério quando técnicas de otimização de registradores são usadas (conforme discutido no Capítulo 13), porque essas técnicas de compilação tentam aumentar o uso de registradores, maximizando dessa forma o número de conflitos de armazenamento.

Uma maneira de enfrentar esses tipos de conflitos de armazenamento é baseada em uma solução tradicional de conflitos de recursos: duplicação de recursos. Neste contexto, a técnica é conhecida como **renomeação de registradores**. Basicamente, registradores são alocados dinamicamente pelo hardware do processador e são associados com valores usados pelas instruções em vários pontos do tempo. Quando um novo valor de registrador é criado (por exemplo, quando uma instrução que tem um registrador como um operando de destino é executado), um novo registrador é alocado para esse valor. As instruções subsequentes que acessam esse valor como operando de origem nesse registrador têm que passar pelo processo de renomeação: as referências de registradores nessas instruções precisam ser revisadas para que se refiram ao registrador que contém o valor necessário. Assim, a mesma referência do registrador original em várias instruções diferentes pode se referir a registradores reais diferentes, se valores diferentes são pretendidos.

Vamos considerar como a renomeação de registradores poderia ser usada no pedaço de código que estávamos analisando:

```
I1: R3b ← R3a op R5a
I2: R4b ← R3b + 1
I3: R3c ← R5a + 1
I4: R7b ← R3c op R4b
```

A referência de registrador sem a subscrição diz respeito à referência lógica de registrador encontrada na instrução. A referência de registrador com subscrição se refere a um registrador de hardware alocado para guardar um novo valor. Quando uma nova alocação é feita para um registrador lógico em particular, as referências das instruções subsequentes para esse registrador lógico com um operando de origem são feitas para se referir ao registrador de hardware alocado mais recentemente (recente em termo de sequência de instruções do programa).

Neste exemplo, a criação do registrador R3_c na instrução I3 evita a dependência RAW da segunda instrução e a dependência de saída da primeira instrução e não interfere no valor correto sendo acessado por I4. O resultado é que I3 pode ser emitida imediatamente; sem renomeação, I3 não pode ser emitida até que a primeira instrução seja completada e a primeira instrução emitida.



Simulador de *scoreboarding*

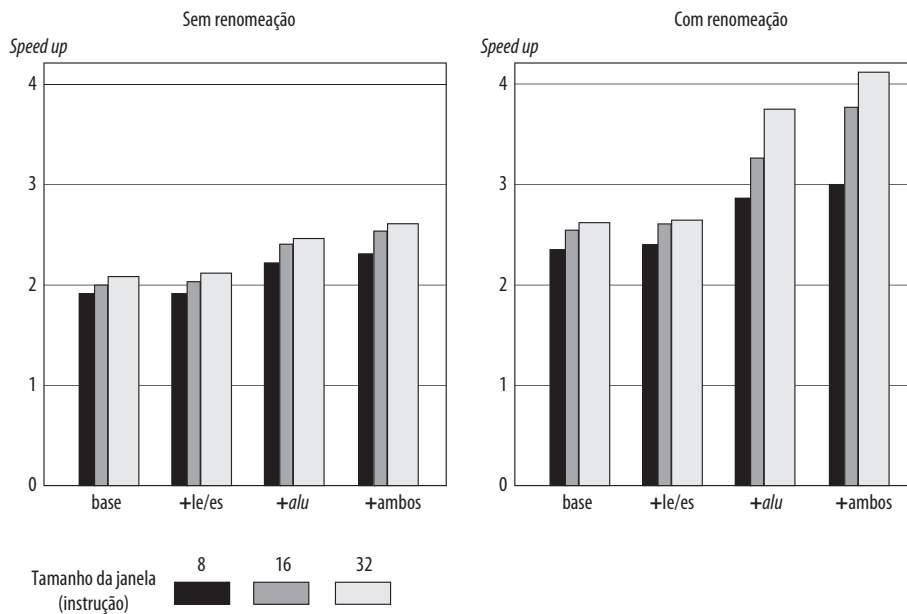
Uma alternativa para renomeação de registradores é um *scoreboarding*. Basicamente, *scoreboarding* é uma técnica de contabilidade que permite que as instruções sejam executadas sempre quando não são dependentes das instruções anteriores e nenhum perigo estrutural está presente. Veja uma discussão sobre o assunto no Apêndice I.



Paralelismo de máquinas

Analisamos anteriormente três técnicas de hardware que podem ser usadas em um processador superescalar para melhorar o desempenho: duplicação de recursos, emissão fora-de-ordem e renomeação. Um estudo que esclarece a relação entre essas técnicas foi reportado em Smith, Johnson e Horowitz (1989^h). O estudo usa uma simulação que modelou uma máquina com características do MIPS R2000, acrescida de vários recursos superescalares. Uma série de diferentes sequências de programa foi simulada.

A Figura 14.5 mostra os resultados. Em cada um dos gráficos, o eixo vertical corresponde ao aumento médio de velocidade (*speed up*) da máquina superescalar em relação à máquina escalar. O eixo horizontal mostra os resultados para quatro organizações alternativas de processadores. A máquina base não duplica nenhuma das unidades funcionais, mas pode emitir instruções fora de ordem. A segunda configuração duplica a unidade funcional de

Figura 14.5 Acelerações de várias organizações de máquinas sem dependências procedurais

leitura/escrita que acessa dados da cache. A terceira configuração duplica a ALU e a quarta organização duplica ambos, leitura/escrita e ALU. Em cada gráfico, os resultados são mostrados para tamanhos das janelas de instruções de 8, 16 e 32 instruções, o que define a quantidade de análise antecipada que o processador consegue fazer. A diferença entre os dois gráficos é que, no segundo, a renomeação de registradores é permitida. Isso equivale a dizer que o primeiro gráfico representa uma máquina limitada por todas as dependências, enquanto o segundo corresponde a uma máquina limitada apenas por dependências verdadeiras.

Os dois gráficos combinados produzem algumas conclusões importantes. A primeira é que provavelmente não vale a pena adicionar unidades funcionais sem renomeação de registradores. Existe uma pequena melhoria de desempenho, mas a custo de aumento da complexidade do hardware. Com a renomeação de registradores, que elimina antidependências e dependências de saída, ganhos notáveis são alcançados ao se adicionarem mais unidades funcionais. Observe, no entanto, que há uma diferença significativa na quantidade de ganhos obtidos entre usar uma janela de instruções de 8 *versus* uma janela de instruções maior. Isso indica que se o tamanho da janela de instruções for pequeno demais, as dependências de dados irão evitar a utilização eficiente de unidades funcionais extras: o processador deve ser capaz de analisar muito à frente para localizar instruções independentes para utilizar o hardware mais completamente.



Simulador de pipeline com agendamento estático *versus* dinâmico



Previsão de desvio

Qualquer máquina de alto desempenho com pipeline deve ter algum tipo de tratamento para lidar com desvios. Por exemplo, o Intel 80486 tratava o problema tanto obtendo a próxima instrução sequencial depois de um desvio, quanto obtendo especulativamente a instrução alvo do desvio. No entanto, como há dois estágios de pipeline entre leitura e execução, esta estratégia provoca um atraso de dois ciclos quando o desvio é tomado.

Com a chegada das máquinas RISC, a estratégia de desvio atrasado foi explorada. Isso permite ao processador calcular o resultado de instruções de desvio condicional antes que qualquer instrução inutilizável seja obtida. Com este método, o processador sempre executa a única instrução que vem imediatamente depois do desvio. Isso mantém o pipeline cheio enquanto o processador obtém um novo fluxo de instruções.

Com o desenvolvimento de máquinas superescalares, a estratégia de desvio atrasado tem menos apelo. O motivo é que múltiplas instruções precisam executar no *delay slot*, trazendo vários problemas relacionados com dependências das instruções. Assim, as máquinas superescalares retornaram às técnicas de previsão de desvios pré-RISC. Algumas, como PowerPC 601, usam uma técnica estática simples de previsão de desvio. Os processadores mais sofisticados, como PowerPC 620 e Pentium 4, usam previsão de desvios dinâmica baseada na análise do histórico de desvios.

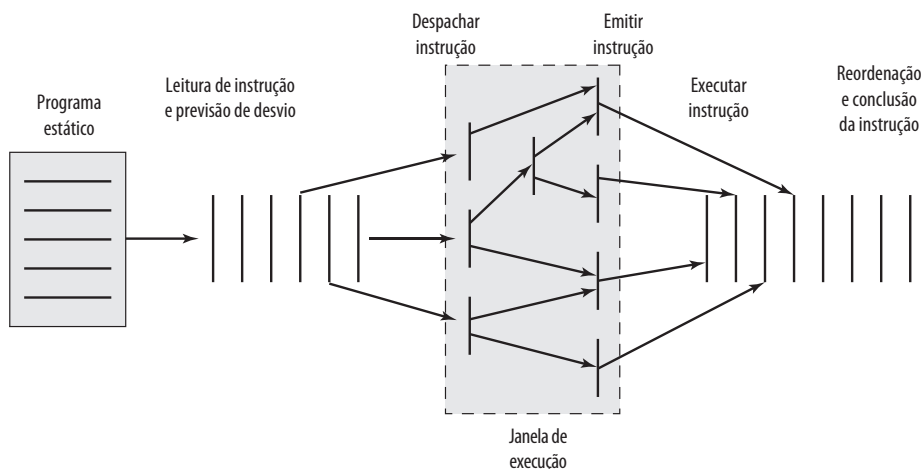


Execução superescalar

Estamos em uma posição agora onde podemos fornecer uma visão de execução superescalar de programas; isso é ilustrado na Figura 14.6. O programa a ser executado consiste de uma sequência linear de instruções. Esse é o programa estático conforme escrito pelo programador ou gerado pelo compilador. O processo de obter instrução, o que inclui a previsão de desvio, é usado para formar um fluxo dinâmico de instruções. Este fluxo é examinado para dependências e o processador pode remover as dependências artificiais. O processador então despacha as instruções para uma janela de execução. Nesta janela, as instruções não formam mais um fluxo sequencial, mas são estruturadas de acordo com suas dependências de dados verdadeiras. O processador efetua o estágio de execução de cada instrução numa ordem determinada pelas dependências de dados verdadeiras e pela disponibilidade de recursos de hardware. Finalmente, as instruções são conceitualmente colocadas de volta na ordem sequencial e seus resultados são reordenados.

O passo final mencionado no parágrafo anterior é conhecido como **concluir** (*commit*) ou **retirar** a instrução. Este passo é necessário pelo seguinte motivo. Por causa do uso de pipelines múltiplos e paralelos, as instruções podem completar em uma ordem diferente daquela mostrada no programa estático. Além disso, o uso da previsão de desvio e da execução especulativa significa que algumas instruções podem completar a execução e depois têm que ser abandonadas porque o desvio que elas representam não foi tomado. Portanto, armazenamento permanente e registradores visíveis ao programa não podem ser atualizados imediatamente quando as instruções completam a execução. Os resultados precisam ser guardados em algum tipo de armazenamento temporário que possa ser usado pelas instruções dependentes e depois são tornados permanentes quando for determinado que o modelo sequencial já tenha executado a instrução.

Figura 14.6 Ilustração conceitual de processamento superescalar





Implementação superescalar

Com base nessa discussão, podemos fazer alguns comentários gerais sobre o hardware de processador necessário para abordagem superescalar. Smith e Sohi (1995^m) listam os seguintes elementos-chave:

- As estratégias de busca de instrução que obtêm simultaneamente várias instruções, frequentemente prevendo os resultados das instruções de desvios condicionais. Estas funções requerem o uso de múltiplos estágios de busca e decodificação e lógica de previsão de desvios.
- Lógica para determinar dependências verdadeiras envolvendo valores de registradores e mecanismos para transferir esses valores para onde eles forem necessários durante a execução.
- Mecanismo para iniciar, ou emitir, múltiplas instruções em paralelo.
- Recursos para execução paralela de múltiplas instruções, incluindo múltiplas unidades funcionais de pipeline e hierarquias de memória capazes de atender simultaneamente várias referências de memória.
- Mecanismos para concluir o estado do processo na ordem correta.

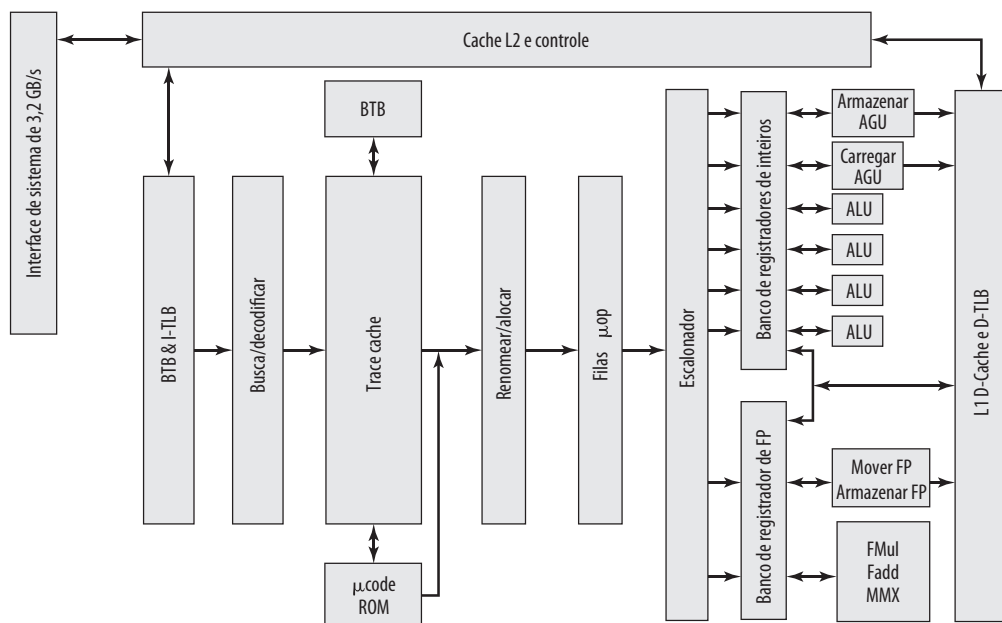


14.3 Pentium 4

Embora o conceito de projeto superescalar seja geralmente associado à arquitetura RISC, os mesmos princípios superescalares podem ser aplicados a uma máquina CISC. Talvez o exemplo mais notável disso seja o Pentium. A evolução de conceitos superescalares na linha Intel é interessante para ser observada. O 386 é uma máquina CISC tradicional sem pipeline. O 486 introduz o primeiro processador x86 com pipeline, reduzindo a latência média de operações de inteiros de dois a quatro ciclos para um ciclo, mas é ainda limitada a executar uma única instrução a cada ciclo, sem elementos superescalares. O Pentium original tinha um componente superescalar modesto, consistindo de uso de duas unidades de execução de inteiros. O Pentium Pro introduziu um design superescalar completo. Modelos Pentium subsequentes refinaram e melhoraram o projeto superescalar.

Um diagrama de bloco geral de Pentium 4 é mostrado na Figura 4.18. A Figura 14.7 ilustra a mesma estrutura em uma maneira mais adequada para discussão sobre pipeline. A operação de Pentium 4 pode ser resumida do seguinte modo:

Figura 14.7 Diagrama de blocos do Pentium 4



AGU= unidade de geração de endereço (do inglês *address generation unit*)

BTB = buffer de alvo de desvio (do inglês *branch target buffer*)

FP = pontos flutuantes (do inglês *floating points*)

D-TLB = TLB de dados

I-TLB = TLB de instruções

1. O processador busca a instrução na memória na ordem do programa estático.
2. Cada instrução é traduzida em uma ou mais instruções RISC de tamanho fixo, conhecidas como **micro-operações** ou **micro-ops**.
3. O processador executa as micro-ops em uma organização de pipeline superescalar para que as micro-ops possam ser executadas fora de ordem.
4. O processador submete o resultado de cada execução de micro-op para o conjunto de registradores do processador na ordem do fluxo do programa original.

Na verdade, a arquitetura do Pentium 4 consiste de uma casca exterior CISC com um núcleo RISC. As micro-ops RISC internas passam por um pipeline de pelo menos 20 estágios (Figura 14.8); em alguns casos, a micro-op requer múltiplos estágios de execução, resultando em um pipeline ainda maior. Isso contraria o pipeline de cinco estágios (Figura 12.21) usado em processadores Intel x86 anteriores e em Pentium.

Nós seguiremos agora o pipeline de Pentium 4, usando a Figura 14.9 para ilustrar sua operação.



Front end

GERAÇÃO DE MICRO-OPS A organização de Pentium 4 inclui um *front end in-order* (Figura 14.9a) que pode ser considerado fora do escopo do pipeline ilustrado na Figura 14.8. Este *front end* alimenta uma cache de instruções L1, chamado de trace cache, que é onde o pipeline começa de fato. Normalmente, o processador opera a partir da trace cache; quando ocorre uma falha na trace cache, o front end *in-order* coloca novas instruções na trace cache.

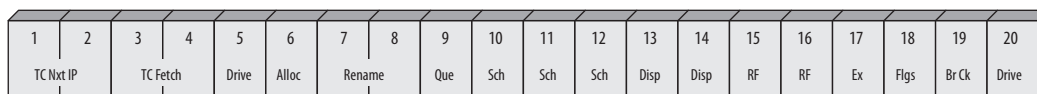
Com a ajuda do buffer de alvo do desvio e do buffer de análise antecipada (BTB e I-TLB), a unidade de busca/decodificação obtém as instruções de máquina de Pentium 4 a partir da cache L2, 64 bytes por vez. Por padrão, as instruções são obtidas sequencialmente para que cada linha obtida da cache L2 inclua a próxima instrução a ser lida. A previsão de desvio através da unidade BTB e I-TLB pode alterar essa operação sequencial de leitura. O I-TLB traduz o endereço do ponteiro linear da instrução dado para o endereço físico requerido para acessar a cache L2. A previsão de desvio estática no *front end* BTB é usada para determinar quais instruções obter a seguir.

Uma vez obtidas as instruções, a unidade de busca/decodificação procura pelos bytes para determinar os limites da instrução; isso é uma operação necessária por causa do tamanho variável das instruções x86. O decodificador traduz cada instrução de máquina para um até quatro micro-ops, cada uma sendo uma instrução RISC de 118 bits. Observe, para efeitos de comparação, que as máquinas RISC mais puras possuem um tamanho da instrução 32 bits. O tamanho maior de micro-op é necessário para acomodar as operações complexas de Pentium. Mesmo assim, as micro-ops são mais fáceis de gerenciar do que as instruções originais das quais derivam.

As micro-ops geradas são guardadas na trace cache.

PONTEIRO DA PRÓXIMA INSTRUÇÃO DA TRACE CACHE Os dois primeiros estágios do pipeline (Figura 14.9b) lidam com seleção de instruções na *trace cache* e envolvem um mecanismo de previsão de desvios separado daquele descrito na seção anterior. O Pentium 4 usa uma estratégia dinâmica de previsão de desvio baseada no histórico de execuções recentes de instruções de desvio. Um buffer de alvo do desvio (BTB) é mantido para guardar as informações de cache sobre instruções de desvio recém-encontradas. Sempre que é encontrada uma instrução de desvio no fluxo de instruções, o BTB é verificado. Se uma entrada já existir em BTB, então a unidade de instrução

Figura 14.8 Pipeline de Pentium 4



TC Nxt IP = ponteiro da próxima instrução da trace cache
 TC Fetch = busca na trace cache
 Alloc = alocar
 Rename = renomeação de registrador

Que = fila micro-op
 Sch = escalonamento micro-op
 Disp = despachar
 RF = banco de registradores

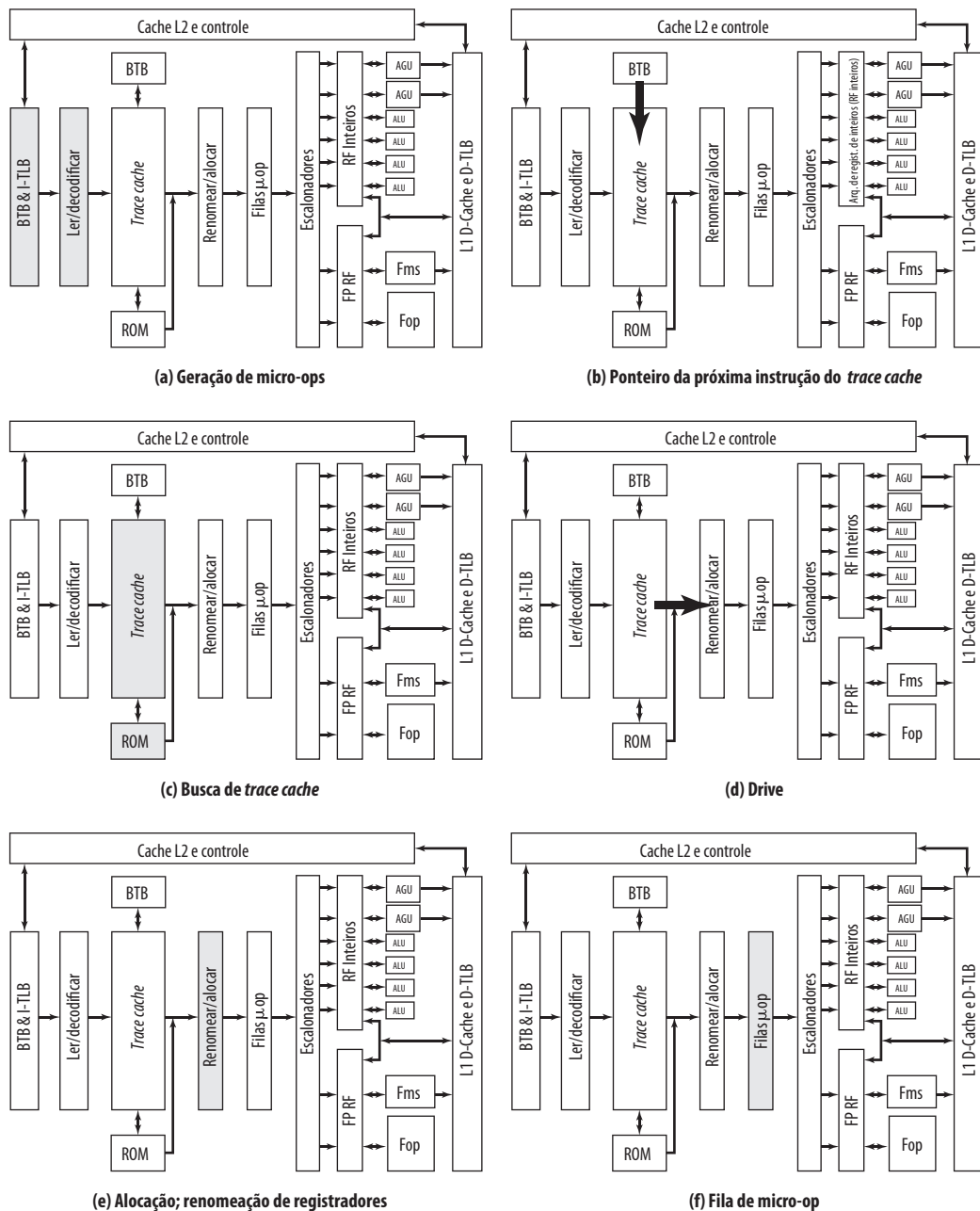
Ex = executar
 Flgs = flags
 Br Ck = verificar desvio

é direcionada pela informação do histórico para que essa entrada determine a previsão sobre a tomada do desvio. Se um desvio for previsto, então o endereço do destino do desvio associado a essa entrada é usado para pré-leitura da instrução alvo do desvio.

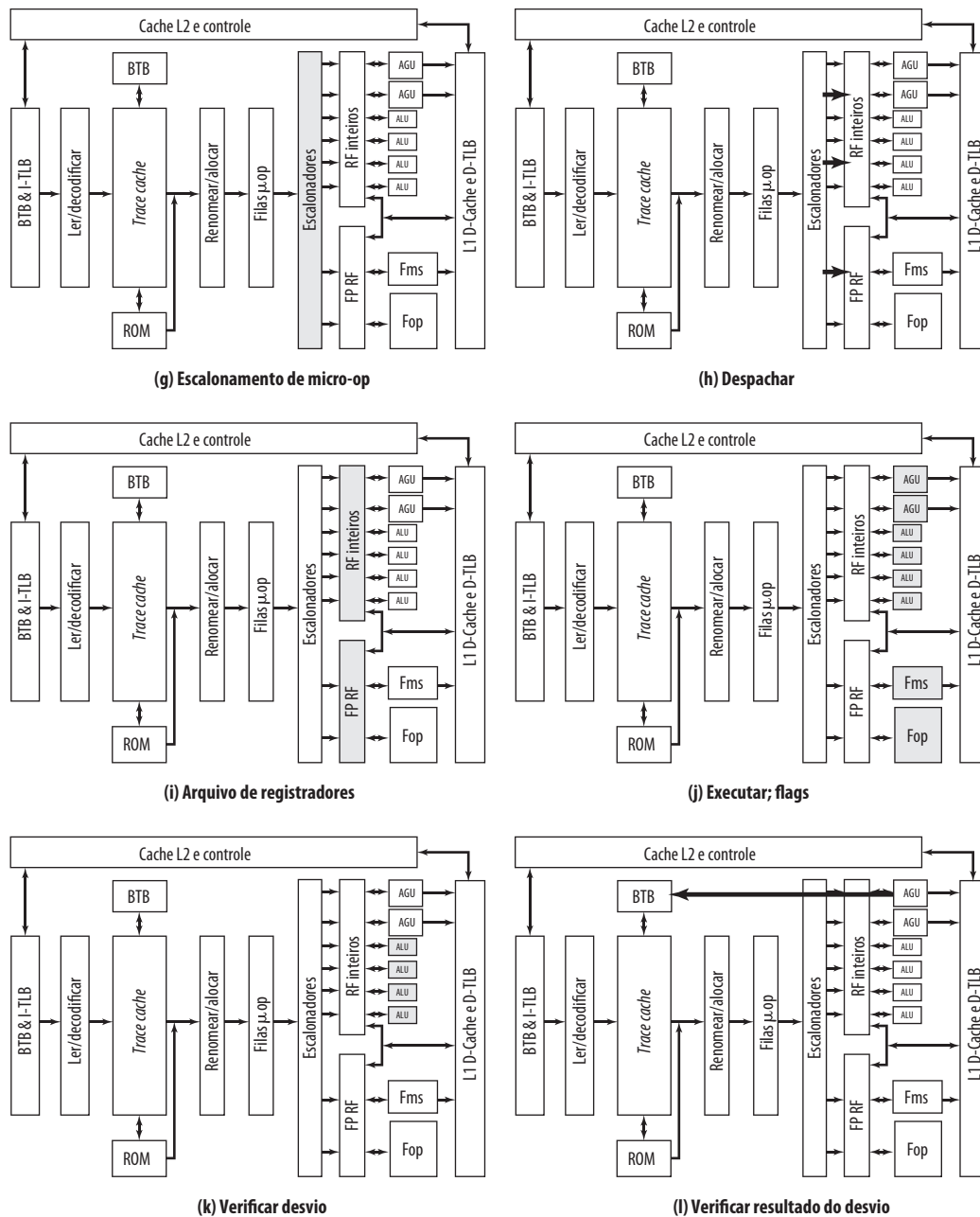
Uma vez executada a instrução, o histórico da entrada apropriado é atualizado para refletir o resultado da instrução de desvio. Se essa instrução não estiver representada em BTB, então o endereço desta instrução é carregado em uma entrada em BTB; se necessário, uma entrada mais antiga é excluída.

A descrição dos dois parágrafos anteriores se encaixa em termos gerais na estratégia de previsão de desvio usada no modelo original de Pentium, assim como em modelos posteriores de Pentium, incluindo o Pentium 4. No entanto, no caso de Pentium, um esquema de histórico relativamente simples de 2 bits é usado. Os modelos de Pentium posteriores possuem pipelines muito maiores (20 estágios para Pentium 4 comparados com 5 estágios

Figura 14.9 Operação do pipeline do Pentium



(continua)

Figura 14.9 Operação do pipeline do Pentium (*continuação*)

para Pentium) e, portanto, a penalidade pelo não acerto é maior. Por causa disso, os modelos de Pentium mais novos usam um esquema de previsão de desvio mais elaborado com mais bits de histórico para reduzir a taxa de previsões falhas.

O BTB do Pentium 4 é organizado como uma cache associativa por conjunto de 4 linhas cada conjunto (*4-way*) com 512 linhas. Cada entrada usa o endereço do desvio. A entrada também inclui o endereço do destino do desvio para última vez em que o desvio foi tomado e um campo de histórico de 4 bits. Esse uso de quatro bits de histórico contraria com 2 bits usados no Pentium original e usados na maioria de processadores superescalares. Com 4 bits, o mecanismo

do Pentium 4 pode levar em conta um histórico maior ao prever desvios. O algoritmo usado é conhecido como algoritmo de Yeh (YEH e PATT, 1991²). Os desenvolvedores desse algoritmo demonstraram que ele provê uma redução significativa de previsões de falhas se comparado aos algoritmos que usam apenas 2 bits de histórico (Evers et al. 1998^o).

Desvios condicionais que não possuem um histórico em BTB são previstos usando um algoritmo estático de previsão, de acordo com as seguintes regras:

- Para endereços de desvio que não são relativos a IP, prever que será tomado se o desvio for um retorno e em outros casos prever que não será tomado.
- Para desvios condicionais relativos a IP para trás, prever que será tomado. Esta regra reflete o comportamento típico de laços de repetição.
- Para desvios condicionais relativos a IP para frente, prever que não será tomado.

BUSCA NA TRACE CACHE A *trace cache* (Figura 14.9c) recebe as micro-ops já decodificadas do decodificador de instruções e as monta em sequências ordenadas de micro-ops (chamadas de *traces*) como no programa. As micro-ops são obtidas sequencialmente a partir da *trace cache*, sujeito a lógica de previsão de desvios.

Poucas instruções requerem mais do que quatro micro-ops. Essas instruções são transferidas para ROM de microcódigo que contém uma série de micro-ops (cinco ou mais) associadas a uma instrução de máquina complexa. Por exemplo, uma instrução de *string* pode ser traduzida em uma sequência muito grande (até centenas) e repetitiva de micro-ops. Assim, a ROM de microcódigo é uma unidade microprogramada conforme discutido na Parte 4. Depois que a ROM de microcódigo terminar de sequenciar as micro-ops para a instrução atual do Pentium, a leitura continua a partir do *trace cache*.

DRIVE O quinto estágio (Figura 14.9d) do pipeline do Pentium 4 entrega instruções decodificadas do *trace cache* para o módulo de renomeação/alocação.



Lógica de execução fora de ordem (*out-of-order execution*)

Esta parte do processador reordena as micro-ops para permitir que elas sejam executadas assim que seus operandos de entrada estiverem prontos.

ALOCAÇÃO O estágio de alocação (Figura 14.9e) aloca os recursos necessários para execução. Ele desempenha as seguintes funções:

- Se um recurso necessário, como um registrador, está indisponível para uma das três micro-ops que estão chegando no alocador durante o ciclo de clock, o alocador para o pipeline.
- O alocador aloca uma entrada no buffer de reordenação (ROB), a qual acompanha o *status* da conclusão de uma das 126 micro-ops que podem estar em processamento a qualquer momento.²
- O alocador aloca um dos 128 registradores de inteiro ou de ponto flutuante para o valor dos dados resultantes da micro-op e possivelmente uma leitura ou uma escrita no buffer usado para acompanhar uma das 48 leituras ou 24 escritas no pipeline de máquina.
- O alocador aloca uma entrada em uma das duas filas de micro-ops na frente do escalonador de instruções.

O ROB é um buffer circular que pode guardar até 126 micro-ops e também contém 128 registradores físicos. Cada entrada do buffer consiste dos campos a seguir:

- **Estado:** indica se a micro-op está escalonada para execução, se foi despachada para execução ou se completou a execução e está pronta para ser retirada.
- **Endereço de memória:** o endereço da instrução Pentium que gerou a micro-op.
- **Micro-op:** a operação atual.
- **Registrador apelido (*alias register*):** se a micro-op referencia um dos 16 registradores arquiteturais, esta entrada redireciona essa referência para um dos 128 registradores de hardware.

As micro-ops entram no ROB na ordem. Elas são, então, despachadas do ROB para unidade de despacho/execução fora de ordem. O critério para despacho é que a unidade de execução apropriada e todos os itens de dados requeridos para essa micro-op estejam disponíveis. Finalmente, as micro-ops são retiradas do ROB na ordem. Para conseguir a retirada em ordem, as micro-ops são retiradas em ordem das mais antigas primeiro depois que cada micro-op foi definida como pronta para retirada.

² Veja Apêndice I para uma discussão sobre buffers de reordenação.

RENOMEAÇÃO DE REGISTRADORES O estágio de renomeação (Figura 14.9e) remapeia as referências a 16 registradores arquiteturais (8 registradores de ponto flutuante mais EAX, EBX, ECX, EDX, ESI, EDI, EBP e ESP) para um conjunto de 128 registradores físicos. O estágio remove falsas dependências causadas por um número limitado de registradores arquiteturais enquanto preserva as dependências de dados verdadeiras (leituras após escritas).

FILAS DE MICRO-OPS Depois da alocação de recursos e renomeação de registradores, as micro-ops são colocadas em uma das duas filas de micro-ops (Figura 14.9f), onde são mantidas até que haja espaço nos escalonadores. Uma das duas filas é para operações de memória (leituras e escritas) e a outra é para micro-ops que não envolvem referências de memória. Cada fila obedece à disciplina FIFO (primeiro a entrar, primeiro a sair), mas nenhuma ordem é mantida entre as filas. Isto é, uma micro-op pode ser lida a partir de uma fila fora de ordem em relação às micro-ops da outra fila. Isto oferece uma flexibilidade maior para escalonadores.

ESCALONAMENTO DE DESPACHO DE MICRO-OPS Os escalonadores (Figura 14.9g) são responsáveis por obter micro-ops a partir das filas de micro-ops e despachá-las para execução. Cada escalonador procura por micro-ops cujo *status* indica que a micro-op possui todos os seus operandos. Se a unidade de execução requerida por essa micro-op estiver disponível, então o escalonador obtém a micro-op e despacha-a para unidade de execução apropriada (Figura 14.9h). Até seis micro-ops podem ser despachadas em um ciclo. Se mais de uma micro-op estiver disponível para uma determinada unidade de execução, então o escalonador as despacha na sequência da fila. Este é um tipo de disciplina FIFO que favorece a execução em ordem, porém, nesse ponto, o fluxo de instrução foi tão rearranjado pelas dependências e desvios que está substancialmente fora de ordem.

Quatro portas ligam os escalonadores às unidades de execução. A porta 0 é usada para instruções de inteiros e as de ponto flutuante, com exceção de operações de inteiros simples e de tratamento de falhas de previsão de desvios, os quais são alocados para a Porta 1. Além disso, as unidades de execução MMX são alocadas entre essas duas portas. As portas restantes são para leituras e escritas de memória.



Unidades de execução de inteiros e de pontos flutuantes

Os bancos de registradores de inteiros e de ponto flutuante são a origem das operações pendentes para unidades de execução (Figura 14.9i). Estas obtêm valores a partir dos bancos de registradores assim como a partir da cache de dados L1 (Figura 14.9j). Um estágio separado do pipeline é usado para calcular os flags (por exemplo, zero, negativo); essas são normalmente a entrada para uma instrução de desvio.

Um estágio de pipeline subsequente efetua verificação de desvios (Figura 14.9k). Esta função compara o resultado atual do desvio com a previsão. Se a previsão de desvio falhar, então há micro-ops em vários estágios do processamento que precisam ser removidas do pipeline. O destino do desvio apropriado é então enviado para o preditor de desvios durante o estágio drive (Figura 14.9l), o qual reinicia o pipeline inteiro a partir do novo endereço do alvo.

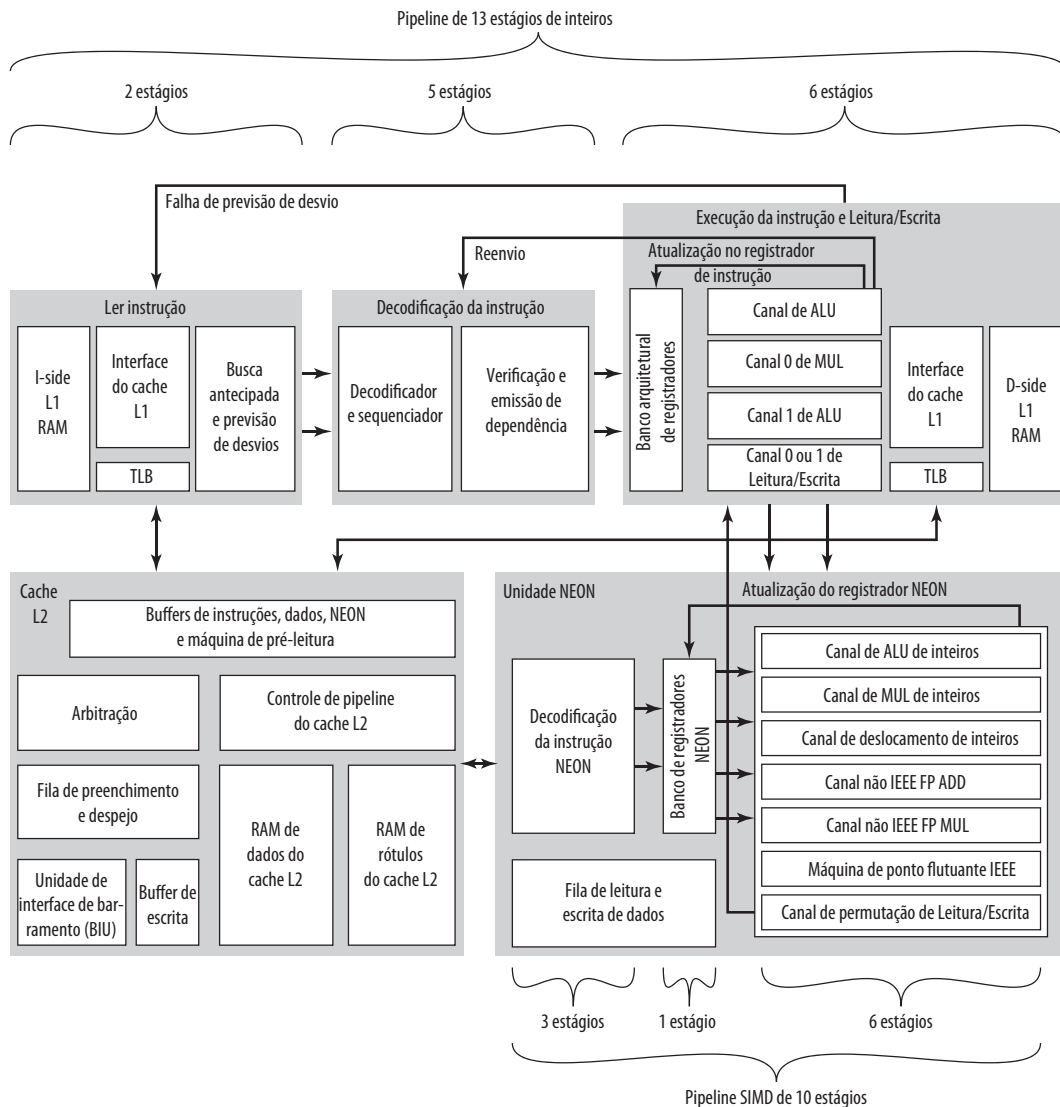


14.4 ARMCortex-A8

Implementações recentes da arquitetura ARM viram a introdução de técnicas superescalares para o pipeline de instruções. Nesta seção, focamos no ARM Cortex-A8, o qual oferece um bom exemplo de um projeto RISC superescalares.

O Cortex-A8 é um processador da família ARM, referido como processador de aplicação. Um processador de aplicações ARM é um processador embarcado que executa sistemas operacionais complexos para aplicações sem fio, de eletrônica de consumo e de imagens. Cortex-A8 têm por alvo uma grande variedade de aplicações móveis e consumidoras incluindo telefones móveis, caixas set-top, consoles de jogos e navegação de automóveis/sistemas de entretenimento.

A Figura 14.10 mostra uma visão lógica da arquitetura do Cortex-A8, enfatizando o fluxo das instruções dentro das unidades funcionais. O fluxo principal das instruções se dá por meio de três unidades funcionais que implementam um pipeline duplo de 13 estágios e com emissão em ordem. Os projetistas de Cortex decidiram usar emissão em ordem para manter a potência adicional reduzida a mínimo. Emissões fora-de-ordem e retirada podem requerer quantidades grandes de lógica que consome potência extra.

Figura 14.10 Diagrama de blocos da arquitetura do ARM Cortex-A8

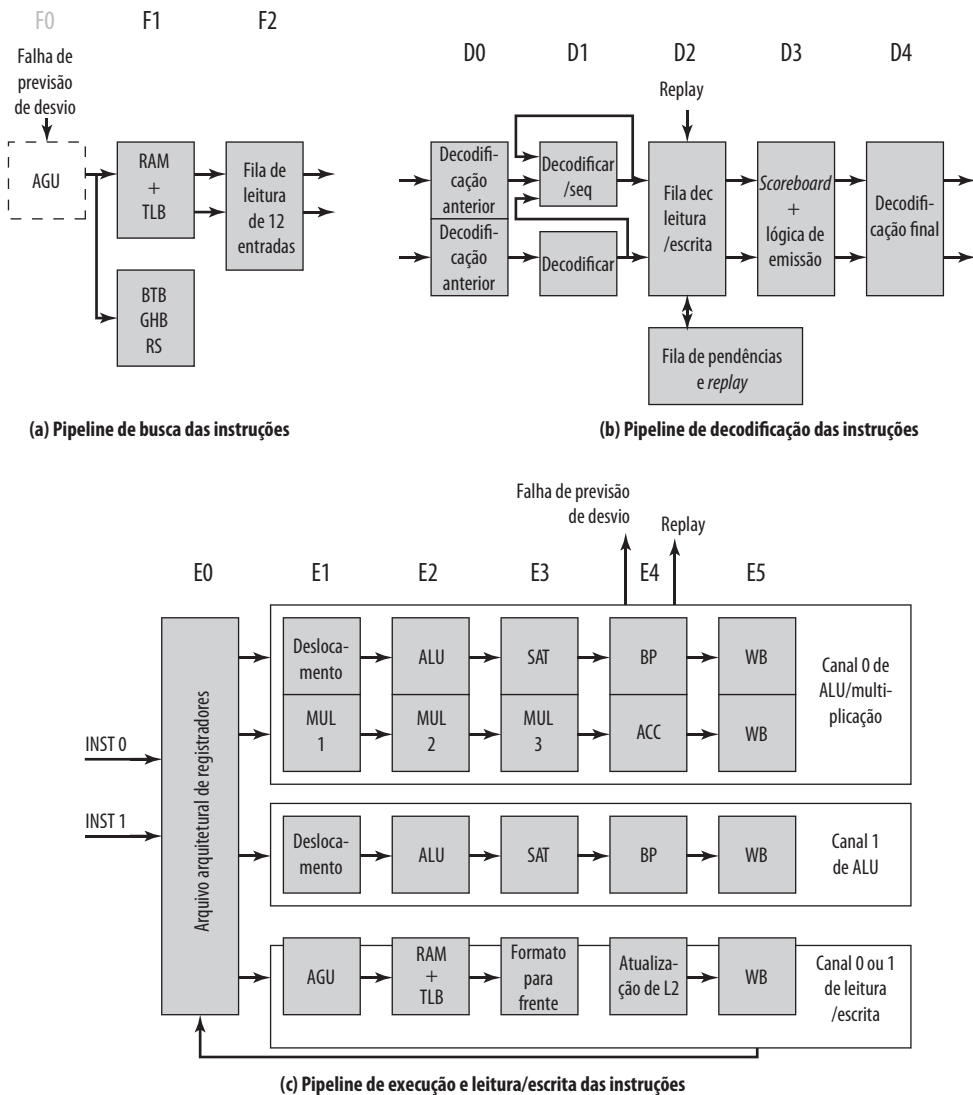
A Figura 14.11 mostra os detalhes do pipeline principal do Cortex-A8. Existe uma unidade separada para SIMD (única-instrução-múltiplos-dados) que implementa um pipeline de 10 estágios.



Unidade de busca das instruções

A unidade de leitura de instruções prevê o fluxo de instruções, obtém instruções da cache L1 de instruções e coloca as instruções obtidas em um buffer para serem consumidas pelo pipeline de decodificação. A unidade de leitura de instruções inclui também a cache L1 de instruções. Como pode haver vários desvios não resolvidos no pipeline, as leituras das instruções são especulativas, o que significa que não há garantia que elas serão executadas. Um desvio ou uma instrução excepcional no fluxo de código pode causar o esvaziamento do pipeline, descartando instruções lidas. A unidade de busca das instruções pode ler até quatro instruções por ciclo e passa pelos seguintes estágios:

FO A unidade de geração de endereços (AGU) gera um novo endereço virtual. Normalmente, este endereço é o próximo endereço na sequência do endereço obtido anteriormente. O endereço também pode ser um endereço

Figura 14.11 Pipeline de inteiros do ARM Cortex-A8

de alvo do desvio fornecido por uma previsão de desvio para uma instrução anterior. F0 não é contado como parte do pipeline de 13 estágios, porque os processadores ARM têm tradicionalmente definido como primeiro estágio o acesso à cache de instruções.

F1 O endereço calculado é usado para obter instruções da cache de instruções L1. Em paralelo, a leitura de endereço é usada para acessar o vetor de previsões de desvio para determinar se a próxima leitura de endereço deve ser baseada em uma previsão de desvio.

F3 Dados da instrução são colocados na fila de instruções. Se uma instrução resultar em previsão de desvio, o novo endereço-alvo é enviado para unidade de geração de endereços.

Para minimizar as penalidades de desvios normalmente associadas com um pipeline mais profundo, o processador Cortex-A8 implementa um predictor de desvios com histórico global de dois níveis, consistindo do buffer de alvos de desvios (BTB) e buffer de histórico global (GHB). Essas estruturas de dados são acessadas em paralelo com a busca das instruções. O BTB indica se a leitura do endereço atual vai ou não retornar uma instrução de desvio e o endereço do seu alvo de desvio. Ele contém 512 entradas. Em um passo apenas no BTB, um desvio é previsto e GHB é acessado. O GHB consiste de 4.096 contadores de 2 bits que codificam a

força e a informação da direção de desvios. Ele é indexado por um histórico de 10 bits da direção de dez últimos desvios encontrados e 4 bits de PC. Além do predictor dinâmico de desvios, uma pilha de retorno é usada para prever os endereços de retorno das sub-rotinas. A pilha de retorno tem oito entradas de 32 bits que armazenam o valor de registrador de ligação em r14 e o estado ARM ou Thumb da função que fez a chamada. Quando é feita uma previsão de que uma instrução do tipo retorno será tomada, a pilha de retorno fornece o último endereço e estado empilhado.

A unidade busca das de instruções pode obter e enfileirar até 12 instruções. Ela emite duas instruções para a unidade de decodificação por vez. A fila possibilita que a unidade de leitura de instruções faça pré-leitura do restante do pipeline de inteiros e crie uma reserva de instruções prontas para decodificação.



Unidade de decodificação de instruções

A unidade de decodificação de instruções decodifica e sequencia todas as instruções ARM e Thumb. Ela possui uma estrutura de pipeline dupla, chamada de **canal0** e **canal1**, para que duas instruções possam passar pela unidade ao mesmo tempo. Quando duas instruções são emitidas a partir do pipeline de decodificação de instruções, o canal0 sempre conterá a instrução mais antiga na ordem do programa. Isto significa que se a instrução no canal0 não puder ser emitida, então a instrução no canal1 não será emitida. Todas as instruções emitidas são processadas na ordem pelo pipeline de execução com resultados sendo atualizados no banco de registradores ao final do pipeline de execução. Esta emissão e retirada em ordem das instruções previne perigos do tipo WAR e mantém diretos o acompanhamento de hazards do tipo WAW e a recuperação das condições de esvaziamento. Assim, a preocupação principal do pipeline de decodificação de instruções é a prevenção de hazards RAW.

Cada instrução passa por cinco estágios de processamento.

D0 Instruções Thumb são descompactadas em instruções ARM de 32 bits. Uma função de decodificação preliminar é efetuada.

D1 A função de decodificação de instruções é completada.

D2 Este estágio escreve instruções em e lê instruções da estrutura de fila pendente/*replay*.

D3 Este estágio contém a lógica de agendamento de instruções. Um *scoreboard* prevê a disponibilidade de registradores usando técnicas de agendamento estáticas.³ A verificação de perigos é feita também neste estágio.

D4 Efetua a decodificação final para todos os sinais de controle requeridos pelas unidades de execução de inteiros e de leitura/escrita.

Nos dois primeiros estágios são determinados o tipo de instrução, operandos de origem e destino e requisitos de recursos para instrução. Algumas instruções normalmente pouco usadas são referidas como instruções de multiciclo. O estágio D1 quebra essas instruções em múltiplos *opcodes* de instrução que são sequenciados individualmente pelo pipeline de execução.

A fila pendente serve para dois propósitos. Primeiro, ela previne que um sinal de parada de D3 circule pelo resto do pipeline. Segundo, ao colocar as instruções em um buffer, sempre deveria haver duas instruções disponíveis para o pipeline duplo. Quando apenas uma instrução é emitida, a fila pendente possibilita que duas instruções prossigam pelo pipeline juntas, mesmo que originalmente tenham sido enviadas da unidade de leitura em ciclos diferentes.

A operação de reenvio é projetada para lidar com os efeitos do sistema de memória sobre o tempo da instrução. As instruções são agendadas estaticamente no estágio D3 com base numa previsão de quando o operando de origem estará disponível. Qualquer atraso do sistema de memória pode resultar em um atraso mínimo de 8 ciclos. Este atraso mínimo de 8 ciclos é equilibrado com o número mínimo de ciclos possíveis para receber dados da cache L2 caso ocorra alguma falha na leitura de L1. A Tabela 14.2 mostra os casos mais comuns que podem resultar em um replay de instruções por causa de um atraso da memória do sistema.

Para lidar com esses atrasos, um mecanismo de recuperação é usado para esvaziar todas as instruções subsequentes no pipeline de execução e reemitir-las (reenvio). Para suportar atrasos, as instruções são copiadas para fila de reenvio antes de serem emitidas e removidas. Se um sinal de reenvio é emitido, as instruções são obtidas da fila de reenvio e entram no pipeline novamente.

³ Veja Apêndice I para uma discussão sobre *scoreboarding*.

Tabela 14.2 Efeitos do sistema de memória de Cortex-A8 no tempo de instruções

Evento de reenvio	Atraso	Descrição
Falha na leitura de dados	8 ciclos	1. Uma falha na leitura da instrução na cache de dados L1. 2. Uma requisição é feita então para a cache de dados L2. 3. Se a falha ocorrer também na cache de dados L2, então ocorre um segundo reenvio. O número de ciclos de atraso depende do tempo do sistema de memória externo. O tempo mínimo necessário para receber a palavra crítica no caso de uma falha de cache L2 é de aproximadamente 25 ciclos, mas pode ser bem maior por causa das latências da memória L3.
Falha de dados TLB	24 ciclos	1. Uma passada pela tabela porque uma falha em L1 TLB causa um atraso de 24 ciclos, assumindo que as entradas da tabela de tradução sejam encontradas na cache L2. 2. Se as entradas da tabela de tradução não estiverem presentes na cache L2, o número de ciclos de atraso depende do tempo do sistema de memória externa.
Buffer de armazenamento cheio	8 ciclos mais a latência para liberar buffer	1. Uma falha no armazenamento da instrução não resulta em nenhum atraso, a não ser que o buffer de armazenamento esteja cheio. 2. No caso de um buffer de armazenamento cheio, o atraso é de no mínimo 8 ciclos. O atraso pode ser maior se demorar mais para liberar algumas entradas do buffer de armazenamento
Requisição de leitura ou escrita não alinhada	8 ciclos	1. Se o endereço de uma instrução de leitura não é alinhado e o acesso cheio não está contido dentro de um limite de 128 bits, há uma penalidade de 8 ciclos. 2. Se o endereço de uma instrução de escrita não é alinhado e o acesso cheio não está contido dentro de um limite de 64 bits, há uma penalidade de 8 ciclos.

A unidade de execução emite duas instruções em paralelo para unidade de execução, a não ser que encontre uma restrição de emissão. A Tabela 14.3 mostra os casos mais comuns de restrições.



Unidade de execução de inteiros

A unidade de execução de instruções consiste de dois pipelines de unidades lógicas aritméticas (ALU) simétricos, um gerador de endereços para ler e armazenar instruções e um pipeline de multiplicação. Os pipelines de execução efetuam também a atualização nos registradores. A unidade de execução de instruções:

- Executa todas as operações de inteiros de ALU e multiplicação, incluindo geração de flags.
- Gera os endereços virtuais para leituras e escritas e o valor base de retorno, quando requerido.
- Fornece dados formatados para escritas e encaminha para frente dados e flags.
- Processa desvios e outras alterações do fluxo de instruções e avalia os códigos condicionais das instruções.

Para instruções da ALU, cada pipeline pode ser usado, consistindo dos seguintes estágios:

E0 Acessar o banco de registradores. Até seis registradores podem ser lidos a partir do arquivo de registradores para duas instruções.

Tabela 14.3 Restrições de emissão dupla do Cortex-A8

Tipo de restrição	Descrição	Exemplo	Ciclo	Restrição
Hazard de leitura/escrita de recursos	Existe apenas um pipeline de LE. Apenas uma instrução de LE pode ser emitida por ciclo. Ela pode estar no pipeline 0 ou pipeline 1	LDR r5, [r6] STR r7, [r8] MOV r9, r10	1 2 2	Esperar pela unidade LE Emissão dupla possível
Hazard de recurso da multiplicação	Existe apenas um pipeline de multiplicação e está disponível apenas no pipeline 0.	ADD r1, r2, r3 MUL r4, r5, r6 MUL r7, r8, r9	1 2 3	Esperar pelo pipeline 0 Esperar pela unidade de multiplicação
Hazard de recurso de desvio	Pode haver apenas um desvio por ciclo. Ele pode estar no pipeline 0 ou no pipeline 1. Um desvio é qualquer instrução que altera PC.	BX r1 BEQ 0x1000 ADD r1, r2, r3	1 2 2	Esperar por desvio Emissão dupla possível
Hazard de saída de dados	Instruções com o mesmo destino não podem ser emitidas no mesmo ciclo. Isto pode acontecer com código condicional.	MOVEQ r1, r2 MOVNE r1, r3 LDR r5, [r6]	1 2 2	Esperar por causa da dependência de saída Emissão dupla possível
Hazard de origem de dados	Instruções não podem ser emitidas se os seus dados não estão disponíveis. Veja as tabelas de escalonamento para origem dos requisitos e os resultados dos estágios.	ADD r1, r2, r3 ADD r4, r1, r6 LDR r7, [r4]	1 2 4	Esperar por r1 Esperar dois ciclos por r4
Instruções multiciclo	Instruções multiciclo devem emitir no pipeline 0 e podem fazer emissão dupla apenas na sua última iteração	MOV r1, r2 LDM r3, {r4-r7} LDM (ciclo 2) LDM (ciclo 3) ADD r8, r9, r10	1 2 3 4 4	Esperar por pipeline 0, Transferir r4 Transferir r5, r6 Transferir r7 Emissão dupla possível na última transferência

E1 O deslocador rápido (Figura 12.25) efetua a sua função, se necessário.

E2 A unidade de ALU (Figura 12.25) efetua a sua função.

E3 Se necessário, este estágio completa saturação aritmética usada por algumas instruções ARM de processamento de dados.

E4 Qualquer alteração no fluxo de controle, incluindo falhas na previsão de desvios, exceções e reenvios do sistema de memória são priorizados e processados.

E5 Resultados das instruções ARM são atualizados no arquivo de registradores.

As instruções que invocam a unidade de multiplicação (Figura 12.25) são encaminhadas para canal0; a operação de multiplicação é feita nos estágios E1 até E3 e a operação de acumulação da multiplicação é feita no estágio E4. Pipeline de leitura/escrita ocorre em paralelo ao pipeline de inteiros. Os estágios são seguintes:

E1 O endereço de memória é gerado a partir do registrador base e indexador.

E2 O endereço é aplicado a vetores da cache.

E3 No caso de uma leitura, os dados são retornados e formatados para serem encaminhados para unidade ALU ou MUL. No caso de uma escrita, os dados são formatados e prontos para serem escritos na cache.

E4 Atualiza a cache L2, se necessário.

E5 Resultados das instruções ARM são escritos de volta no banco de registradores.

A Tabela 14.4 mostra um pedaço de código como exemplo e indica como o processador poderia agendá-lo.

Tabela 14.4 Sequência exemplo de emissão dupla de instruções para pipeline de inteiros no Cortex-A8

Ciclo	Contador de programa	Instrução	Descrição de tempo
1	0x00000ed0	BX r14	Emissão dupla pipeline 0
1	0x00000ee4	CMP r0,#0	Emissão dupla no pipeline 1
2	0x00000ee8	MOV r3,#3	Emissão dupla no pipeline 0
2	0x00000eec	MOV r0,#0	Emissão dupla no pipeline 1
3	0x00000ef0	STREQ r3,[r1,#0]	Emissão dupla no pipeline 0, r3 não necessário até E3
3	0x00000ef4	CMP r2,#4	Emissão dupla no pipeline 1
4	0x00000ef8	LDRLS pc,[pc,r2,LSL #2]	Emissão única no pipeline 0, +1 ciclo para carregar para PC nenhum ciclo extra para deslocamento desde LSL #2
5	0x00000f2c	MOV r0,#1	Emissão dupla com segunda iteração de leitura no pipeline 1
6	0x00000f30	B {pc} + 8	#0xf38 emissão dupla no pipeline 0
7	0x00000f38	STR r0,[r1,#0]	Emissão dupla no pipeline 1
7	0x00000f3c	LDRpc,[r13],#4	Emissão única no pipeline 0, +1 ciclo para carregar para PC
8	0x0000017c	ADD r2,r4,#0xc	Emissão dupla com segunda iteração de leitura no pipeline 1
9	0x00000180	LDR r0,[r6,#4]	Emissão dupla no pipeline 0
9	0x00000184	MOV r1,#0xa	Emissão dupla no pipeline 1
12	0x00000188	LDR r0,[r0,#0]	Emissão única no pipeline 0: r0 produzido em E3, requerido em E1, então +2 ciclos de atraso
13	0x0000018c	STR r0,[r4,#0]	Emissão única no pipeline 0 por causa de perigo de leitura/escrita de recurso, nenhum atraso extra para r0 desde que foi produzido em E3 e consumido em E3
14	0x00000190	LDR r0,[r4,#0xc]	Emissão única no pipeline 0 por causa de perigo de leitura/escrita de recurso
15	0x00000194	LDMFD r13!,{r4- r6, r14}	Leitura de múltiplas leituras de r4 no primeiro ciclo, r5 e r6 no segundo ciclo, r14 no terceiro ciclo, 3 ciclos no total
17	0x00000198	B{pc} + 0xda8	#0xf40 emissão dupla no pipeline 1 com terceiro ciclo de LDM
18	0x00000f40	ADD r0,r0,#2 ARM	Emissão única no pipeline 0
19	0x00000f44	ADD r0,r1,r0 ARM	Emissão única no pipeline 0, nenhuma emissão dupla por causa do perigo em r0 produzido em E2 e requerido em E2



Pipeline SIMD e de ponto flutuante

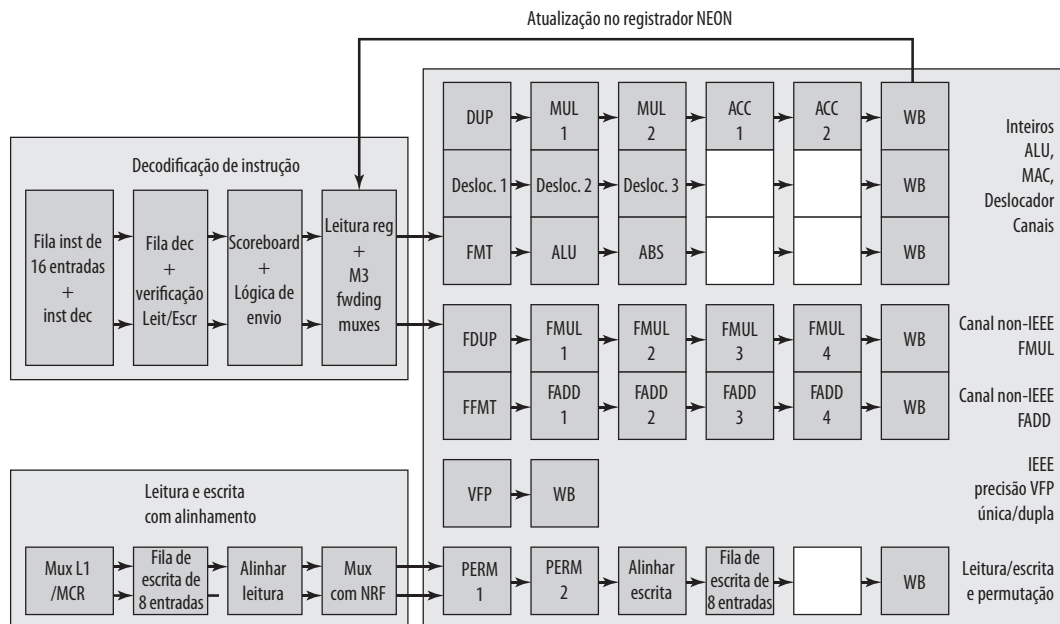
Todas as instruções SIMD e de ponto flutuante passam pelo pipeline de inteiros e são processadas em um pipeline separado de 10 estágios (Figura 14.12). Esta unidade, conhecida como unidade NEON, trata as instruções SIMD empacotadas e fornece dois tipos de suporte para ponto flutuante. Se implementado, um coprocessador vetorial de ponto flutuante (VFP) efetua uma operação de ponto flutuante de acordo com o padrão IEEE 754. Se o coprocessador não estiver presente, então pipelines separados de multiplicação e adição implementam as operações de ponto flutuante.



14.5 Leitura recomendada

Dois bons livros sobre projeto superescalar são Shen e Lipasti (2005⁹) e Omondi (1999⁹). Artigos de pesquisa que valem a pena sobre o assunto são Smith e Sohi (1995^m) e Sima (1997^l). Jouppi e Wall (1989⁹) examinam o paralelismo em nível de instruções, analisam várias técnicas para maximizar paralelismo e comparam abordagem superescalar e superpipeline usando simulação. Artigos recentes que oferecem boa cobertura sobre questões de designs superescalares incluem Sima (2004⁹), Patt (2001^l) e Moshovos e Sohi (2001⁹).

Figura 14.12 Pipeline NEON e de ponto flutuante do ARM Cortex-A8



Popescu et al. (1991)^v fornecem uma análise detalhada sobre uma máquina superescalar proposta, bem como um tutorial excelente sobre questões de projeto relacionadas a políticas de instruções fora-de-ordem. Outra análise de um sistema proposto é encontrada em Kuga, Murakami e Tomita (1991^w); este artigo levanta e considera a maioria das questões de projeto mais importantes para implementação superescalar. Lee, Kwok e Briggs (1991^m) examinam técnicas de software que podem ser usadas para melhorar o desempenho superescalar. Wall (1991^x) é um estudo interessante sobre como o paralelismo em nível de instruções pode ser explorado em um processador superescalar.

O Volume I de Intel (2004^y) provê uma descrição geral do pipeline do Pentium 4; mais detalhes podem ser encontrados em Intel (2001^z) e Intel (2001^{aa}). Outro tratado detalhado é Fog (2008^{bb}).

John e Rubio (2008^{cc}) e ARM (2008^{dd}) fornecem uma cobertura completa sobre pipeline do ARM Cortex-A8. Riches et al. (2007^{ee}) é uma boa visão geral.

Principais termos, perguntas de revisão e problemas

Principais termos

Antidependência	Paralelismo de máquinas	Renomeação de registradores
Previsão de desvios	Micro-operações	Conflito de recursos
Conclusão	Micro-ops	Retirada
Dependência de fluxo	Conclusão fora-de-ordem	Superpipeline
Emissão em-ordem	Emissão fora-de-ordem	Superescalar
Conclusão em-ordem	Dependência de saída	Dependência de dados verdadeira
Emissão de instrução	Dependência procedural	Dependência de leitura-escrita
Paralelismo em nível de instruções	Dependência de leitura-escrita	Dependência de escrita-escrita
Janela de instruções		

Perguntas de revisão

- 14.1** Quais são as características essenciais da abordagem superescalar para projeto de processadores?
- 14.2** Qual é a diferença entre a abordagem superescalar e superpipeline?
- 14.3** O que é paralelismo em nível de instruções?
- 14.4** Defina brevemente os seguintes itens:
- Dependência de dados verdadeira
 - Dependência procedural
 - Conflito de recursos
 - Dependência de saída
 - Antidependência
- 14.5** Qual é a diferença entre paralelismo em nível de instruções e paralelismo de máquina?
- 14.6** Enumere e defina brevemente três tipos de políticas de emissão de instruções superescalar.
- 14.7** Qual é a função de uma janela de instruções?
- 14.8** O que é a renomeação de registradores e qual é a sua função?
- 14.9** Quais são os elementos-chaves da organização de um processador superescalar?

Problemas

- 14.1** Quando a conclusão fora-de-ordem é usada em um processador superescalar, a continuação da execução após um processamento interrompido é complicada, porque condições excepcionais podem ter sido detectadas como uma instrução que produziu seus resultados fora de ordem. O programa não pode ser reiniciado na instrução que segue a instrução de exceção porque instruções subsequentes já completaram e fazendo isso, essas instruções executariam duas vezes. Sugira um mecanismo ou mecanismos para lidar com esta situação.
- 14.2** Considere a seguinte sequência de instruções, onde a sintaxe consiste de um *opcode* seguido por um registrador de destino seguido por um ou dois registradores de origem:

```

0   ADD    R3, R1, R2
1   LOAD   R6, [R3]
2   AND    R7, R5, 3
3   ADD    R1, R6, R0
4   SRL    R7, R0, 8
5   OR     R2, R4, R7
6   SUB    R5, R3, R4
7   ADD    R0, R1, R10
8   LOAD   R6, [R5]
9   SUB    R2, R1, R6
10  AND    R3, R7, 15

```

Suponha o uso de um pipeline de quatro estágios: busca, decodificação/emissão, execução, atualização. Suponha que todos os estágios do pipeline ocupem um ciclo de clock exceto o estágio de execução. Para instruções lógicas e aritméticas de inteiros simples, o estágio de execução ocupa um ciclo, mas para ler da memória cinco ciclos são consumidos no estágio de execução.

Se temos um pipeline escalar simples mas que permita execução fora de ordem, podemos construir a seguinte tabela para execução das sete primeiras instruções:

Instrução	Busca	Decodificação	Execução	Atualização
0	0	1	2	3
1	1	2	4	9
2	2	3	5	6
3	3	4	10	11
4	4	5	6	7
5	5	6	8	10
6	6	7	9	12

As entradas abaixo do pipeline de quatro estágios indicam o ciclo de clock em que cada instrução inicia cada fase. Neste programa, a segunda instrução ADD (instrução 3) depende da instrução LOAD (instrução 1) para um dos seus operandos, r6. Como a instrução LOAD ocupa cinco ciclos

de clock e a lógica de emissão encontra a instrução ADD dependente depois de dois ciclos, a lógica de emissão deve atrasar a instrução ADD para três ciclos de clock. Com a capacidade fora-de-ordem, o processador pode parar a instrução 3 no ciclo de clock 4 e depois continuar para emitir as três instruções independentes a seguir, as quais entram em execução em ciclos 6, 8 e 9. O LOAD termina a execução no ciclo 9 e então o ADD dependente pode ser enviado para execução no ciclo 10.

- Complete a tabela anterior.
- Refaça a tabela supondo que não há capacidade fora-de-ordem. Quais são as economias usando a capacidade?
- Refaça a tabela supondo uma implementação superescalar que pode tratar duas instruções ao mesmo tempo em cada estágio.

14.3 Considere o seguinte programa na linguagem de montagem:

```

I1: Move R3, R7      /R3 ← (R7)/
I2: Load R8, (R3)   /R8 ← Memory (R3)/
I3: Add R3, R3, 4     /R3 ← (R3) + 4/
I4: Load R9, (R3)   /R9 ← Memory (R3)/
I5: BLE R8, R9, L3    /Branch if (R9) > (R8)/

```

Este programa inclui dependências WAW, RAW e WAR. Mostre-as.

14.4 a. Identifique as dependências leitura-escrita, escrita-escrita e leitura-escrita na sequência de instruções a seguir:

```

I1: R1 = 100
I2: R1 = R2 + R4
I3: R2 = R4 - 25
I4: R4 = R1 + R3
I5: R1 = R1 + 30

```

- Renomeie os registradores da parte (a) para prevenir problemas de dependência. Identifique referências para valores iniciais de registradores usando subscrição "a" para referência de registrador.

14.5 Considere a sequência de execução "emissão-em-ordem/conclusão-em-ordem" mostrada na Figura 14.13.

- Identifique o motivo mais provável por que I2 poderia não entrar no estágio de execução até o quarto ciclo. "Emissão em-ordem/conclusão fora-de-ordem" ou "emissão fora-de-ordem/conclusão fora-de-ordem" consertará isso? Se sim, como?
- Identifique o motivo pelo qual I6 não poderia entrar o estágio de escrita até o nono ciclo. "Emissão em-ordem/conclusão fora-de-ordem" ou "emissão fora-de-ordem/conclusão fora-de-ordem" consertará isso? Se sim, como?

14.6 A Figura 14.14 mostra um exemplo de organização de um processador superescalar. O processador pode emitir duas instruções por ciclo se não houver conflito de recursos e problemas de dependência de dados. Existem essencialmente dois pipelines com quatro estágios de processamento (leitura, decodificação, execução e escrita). Cada pipeline possui a sua própria unidade de busca, decodificação e escrita. Quatro unidades funcionais (multiplicador, somador, unidade lógica e unidade de leitura) estão disponíveis para uso no estágio de execução e são compartilhadas pelos dois pipelines de uma maneira dinâmica. As duas unidades de escrita podem ser usadas dinamicamente pelos dois pipelines, dependendo da disponibilidade em um determinado ciclo. Existe uma janela de análise antecipada com sua própria lógica de leitura e decodificação. Esta janela é usada para análise antecipada de instruções para emissão de instruções fora-de-ordem.

Figura 14.13 Uma sequência de execução de emissão em-ordem, conclusão em-ordem

Decodificação		Execução			Escrita		Ciclo
I1	I2						1
	I2			I1			2
	I2			I1			3
I3	I4		I2				4
I5	I6		I4	I3	I1	I2	5
I5	I6	I5		I3			6
		I5	I6		I3	I4	7
							8
					I5	I6	9

Considere o seguinte programa a ser executado nesse processador:

```

I1: Load R1, A      /R1 ← Memory (A)/
I2: Add R2, R1       /R2 ← (R2) + R(1)/
I3: Add R3, R4       /R3 ← (R3) + R(4)/
I4: Mul R4, R5       /R4 ← (R4) + R(5)/
I5: Comp R6          /R6 ← (R6)/
I6: Mul R6, R7       /R3 ← (R3) + R(4)/

```

- Quais dependências existem no programa?
- Mostre a atividade do pipeline para este programa no processador da Figura 14.14 usando políticas de emissão em-ordem e conclusão em-ordem e usando uma apresentação semelhante à Figura 14.2.
- Repita para emissão em-ordem com conclusão fora-de-ordem.
- Repita para emissão fora-de-ordem com conclusão fora-de-ordem.

14.7 A Figura 14.15 é de um artigo sobre projeto superescalar. Explique as três partes da figura e defina w , x , y e z .

14.8 O algoritmo dinâmico de previsão de desvio de Yeh, usado no Pentium 4, é um algoritmo de previsão de desvio de dois níveis. O primeiro nível é o histórico de últimos n desvios. O segundo nível é o comportamento das últimas s ocorrências desse padrão único dos últimos n desvios. Para cada instrução de desvio condicional em um programa, há uma entrada em uma Tabela de Histórico de Desvios (BHT). Cada entrada consiste de n bits correspondendo a n últimas execuções da instrução de desvio, com um 1 se o desvio foi tomado e 0 se não foi tomado. Cada entrada em BHT é indexada em uma Tabela de Padrões (PT) que possui $2n$ entradas, uma para cada padrão possível de n bits. Cada entrada PT consiste de s bits que são usados na previsão de desvios, conforme descrito no Capítulo 12 (Figura 12.19). Quando um desvio condicional é encontrado durante leitura e decodificação de instrução, o endereço da instrução é usado para obter a entrada em BHT apropriada, o que mostra o histórico recente da instrução. Depois, a entrada BHT é usada para obter a entrada PT apropriada para previsão de desvio. Depois que o desvio é executado, a entrada BHT é atualizada e depois a entrada PT apropriada é atualizada.

- Testando o desempenho deste esquema, Yeh tentou cinco esquemas diferentes de previsão, ilustrados na Figura 14.16. Identifique quais três esquemas que correspondem àqueles mostrados nas figuras 12.19 e 12.28. Descreva dois esquemas restantes.
- Com este algoritmo, a previsão não é baseada apenas no histórico recente dessa instrução de desvio em particular. Em vez disso, ela é baseada no histórico recente de todos os padrões de desvios que correspondem ao padrão de n bits na entrada em BHT para essa instrução. Sugira uma base lógica para tal estratégia.

Figura 14.14 Um processador superescalar com pipeline duplo

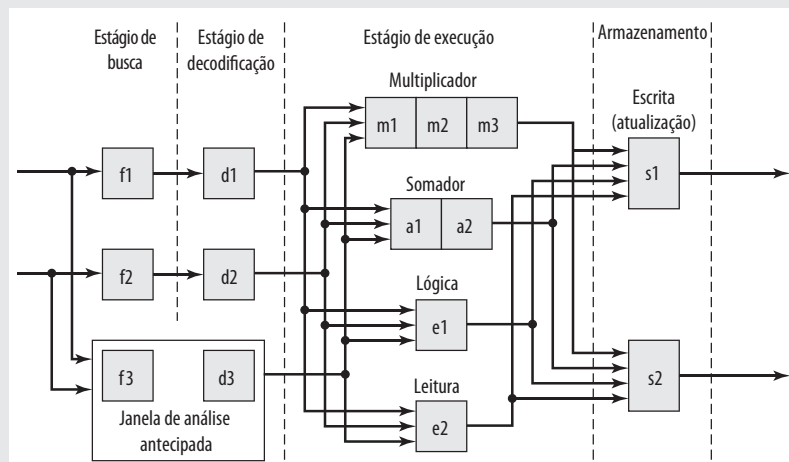
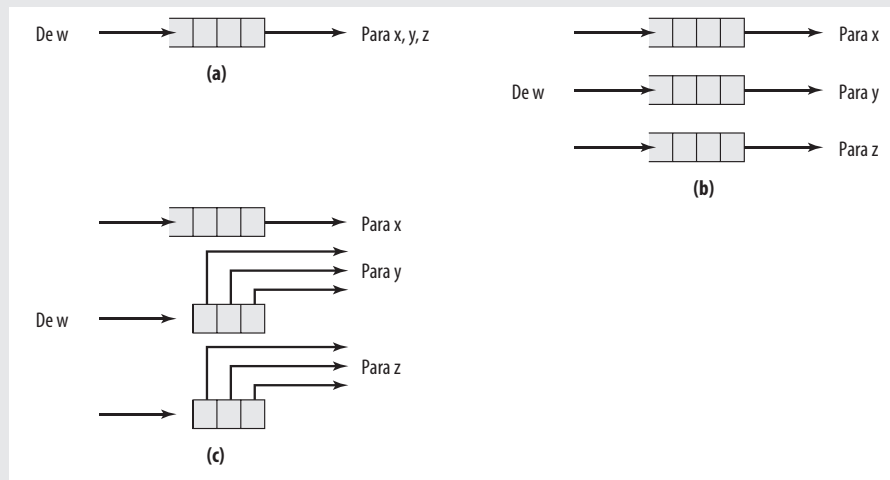
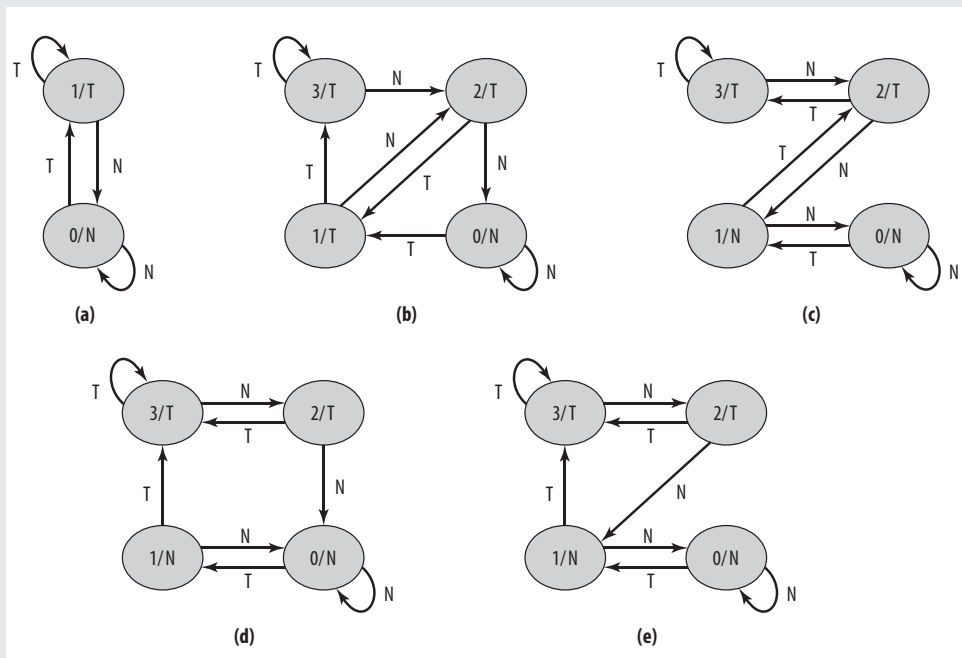


Figura 14.15 Figura para o Problema 14.7**Figura 14.16** Figura para o Problema 14.8

Referências

- a AGERWALA, T. e COCKE, J. *High performance reduced instruction set processors*. Technical Report RC12434 (#55845). Yorktown, NY: IBM Thomas J. Watson Research Center, jan. 1987.
- b JOUPPI, N. "Superscalar versus superpipelined machines". *Computer Architecture News*, jun. 1988.
- c TJADEN, G. e FLYNN, M. "Detection and parallel execution of independent instructions". *IEEE Transactions on Computers*, out. 1970.

- d KUCK, D.; PARKER, D. e SAMEH, A. "An analysis of round methods in floating-point arithmetic". *IEEE Transactions on Computers*, jul. 1977.
- e WEISS, S. e SMITH, J. "Instruction issue logic in pipelined supercomputers". *IEEE Transactions on Computers*, nov. 1984.
- f ACOSTA, R.; KJELSTRUP, J. e TORNG, H. "An instruction issue approach to enhancing performance in multiple functional unit processors". *IEEE Transactions on Computers*, set. 1986.
- g SOHI, G. "Instruction issue logic for high-performance interruptable, multiple functional unit, pipelined computers". *IEEE Transactions on Computers*, mar. 1990.
- h SMITH, M.; JOHNSON, M. e HOROWITZ, M. "Limits on multiple instruction issue". *Proceedings, Third International Conference on Architectural Support for programming Languages and Operating System*, abr. 1989.
- i JOUPPI, N. "The nonuniform distribution of instruction-level and machine parallelism and its effect on performance". *IEEE Transaction on computer*, dez. 1989.
- j LEE, R.; KNOK, A. e BRIGGS, F. "The floating-point performance of a superscalar SPARC processor". *Proceedings, Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, abr. 1991.
- k JOHNSON, M. *Superscalar microprocessor design*. Englewood Cliffs, NJ: Prentice Hall, 1991.
- l JOUPPI, N. e WALL, D. "Available instruction-level parallelism for superscalar and superpipelined machines". *Proceedings, Third International Conference on Architectural Support for Programming Languages and Operating Systems*, abr. 1989.
- m SMITH, J. e SOHI, G. "The microarchitecture of superscalar processors". *Proceedings of the IEEE*, dez. 1995.
- n YEY, T. e PATT, N. "Two-level adapting training branch prediction". *Proceedings, 24th Annual International Symposium on Microarchitecture*, 1991.
- o EVERS, M., et al. "An analysis of correlation and predictability: what makes two-level branch predictors work". *Proceedings, 25th Annual International Symposium on Microarchitecture*, jul. 1998.
- p SHEN, J. e LIPASTI, M. *Modern processor design: fundamentals of superscalar processors*. New York: McGraw-Hill, 2005.
- q OMONDI, A. *The microarchitecture of pipelined and superscalar computers*. Boston: Kluwer, 1999.
- r SIMA, D. "Superscalar instruction issue". *IEEE Micro*, set./out. de 1997.
- s SIMA, D. "Decisive aspects in the evolution of microprocessors". *Proceedings of the IEEE*, dez. 2004.
- t PATT, Y. "Requirements, bottlenecks, and good fortune: agents for microprocessor evolution". *Proceedings of the IEEE*, nov. 2001.
- u MOSHOVOS, A. e SOHI, G. "Microarchitectural innovations: boosting microprocessor performance beyond semiconductor technology scaling". *Proceedings of the IEEE*, nov. 2001.
- v POPESCU, V., et al. "The metaflow architecture". *IEEE Micro*, jun. 1991.
- w KUGA, M.; MURAKAMI, K. e TOMITA, S. "DSNS (dynamically-hazard resolved, statically-code-scheduled, nonuniform superscalar): yet another superscalar processor architecture". *Computer Architecture News*, jun. 1991.
- x WALL, D. "Limits of instruction-level parallelism". *Proceedings, Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, abr. 1991.
- y INTEL CORP. *IA-32 Intel architecture software developer's manual (4 volumes)*. Documento 253665 a 253668. 2004. Disponível em: <<http://developer.intel.com/design/Pentium4/documentation.htm>>.
- z INTEL CORP. *Intel Pentium 4 processor optimization reference manual*. Document 248966-04 2001. Disponível em: <<http://developer.intel.com/design/Pentium4/documentation.htm>>.
- aa INTEL CORP. *Desktop performance and optimization for Intel Pentium 4 Processor*. Documento 248966-04 2001. Disponível em: <<http://developer.intel.com/design/Pentium4/documentation.htm>>.
- bb FOG, A. *The microarchitecture of Intel and AMD CPUs*. Copenhagen University College of Engineering, 2008. Disponível em: <www.agner.org/optimize/>.
- cc JOHN, E. e RUBIO, J. *Unique chips and systems*. Boca Raton, FL: CRC Press, 2008.
- dd ARM Limited. *Cortex-A8 technical reference manual*. ARM DDI 0344E, 2008. Disponível em: <www.arm.com>.
- ee RICHES, S., et al. "A fully automated high performance implementation of ARM Cortex-A8". *IQ Online*, Vol. 6, No. 3, 2007. Disponível em: <www.arm.com/iqonline>.